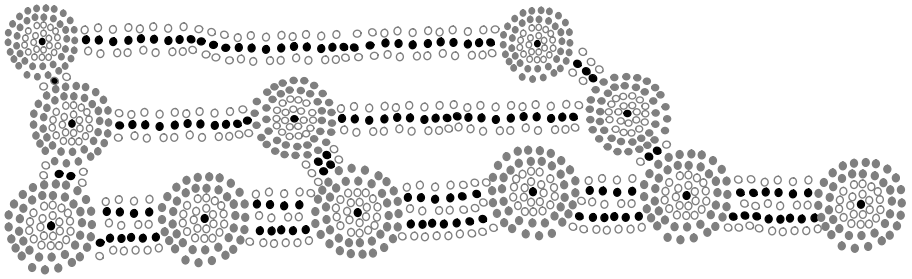
Chapter 10 Maps, Hash Tables, and Skip Lists



Contents

10.1 Maps and Dictionaries 402
10.1.1 The Map ADT 403
10.1.2 Application: Counting Word Frequencies 405
10.1.3 Python's MutableMapping Abstract Base Class 406
10.1.4 Our MapBase Class 407
10.1.5 Simple Unsorted Map Implementation 408
10.2 Hash Tables 410
10.2.1 Hash Functions 411
10.2.2 Collision-Handling Schemes 417
10.2.3 Load Factors, Rehashing, and Efficiency 420
10.2.4 Python Hash Table Implementation 422
10.3 Sorted Maps 427
10.3.1 Sorted Search Tables 428
10.3.2 Two Applications of Sorted Maps 434
10.4 Skip Lists 437
10.4.1 Search and Update Operations in a Skip List 439
10.4.2 Probabilistic Analysis of Skip Lists ★ 443
10.5 Sets, Multisets, and Multimaps 446
10.5.1 The Set ADT 446
10.5.2 Python's MutableSet Abstract Base Class 448
10.5.3 Implementing Sets, Multisets, and Multimaps 450
10.6 Exercises 452

10.1 Maps and Dictionaries

Python's **dict** class is arguably the most significant data structure in the language. It represents an abstraction known as a ***dictionary*** in which unique ***keys*** are mapped to associated ***values***. Because of the relationship they express between keys and values, dictionaries are commonly known as ***associative arrays*** or ***maps***. In this book, we use the term *dictionary* when specifically discussing Python's dict class, and the term *map* when discussing the more general notion of the abstract data type.

As a simple example, Figure 10.1 illustrates a map from the names of countries to their associated units of currency.

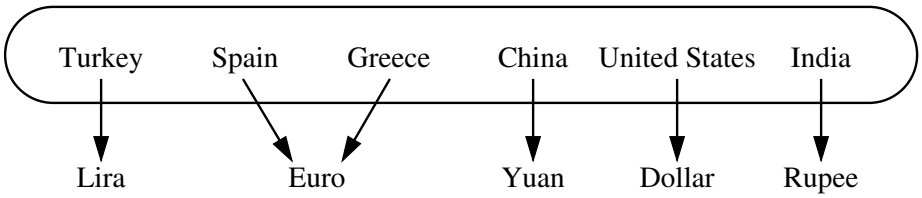


Figure 10.1: A map from countries (the keys) to their units of currency (the values).

We note that the keys (the country names) are assumed to be unique, but the values (the currency units) are not necessarily unique. For example, we note that Spain and Greece both use the euro for currency. Maps use an array-like syntax for indexing, such as currency['Greece'] to access a value associated with a given key or currency['Greece'] = 'Drachma' to remap it to a new value. Unlike a standard array, indices for a map need not be consecutive nor even numeric. Common applications of maps include the following.

- A university's information system relies on some form of a student ID as a key that is mapped to that student's associated record (such as the student's name, address, and course grades) serving as the value.
- The domain-name system (DNS) maps a host name, such as www.wiley.com, to an Internet-Protocol (IP) address, such as 208.215.179.146.
- A social media site typically relies on a (nonnumeric) username as a key that can be efficiently mapped to a particular user's associated information.
- A computer graphics system may map a color name, such as 'turquoise', to the triple of numbers that describes the color's RGB (red-green-blue) representation, such as (64,224,208).
- Python uses a dictionary to represent each namespace, mapping an identifying string, such as 'pi', to an associated object, such as 3.14159.

In this chapter and the next we demonstrate that a map may be implemented so that a search for a key, and its associated value, can be performed very efficiently, thereby supporting fast lookup in such applications.

10.1.1 The Map ADT

In this section, we introduce the ***map ADT***, and define its behaviors to be consistent with those of Python's built-in dict class. We begin by listing what we consider the most significant five behaviors of a map *M* as follows:

M[k]: Return the value v associated with key k in map M, if one exists; otherwise raise a KeyError. In Python, this is implemented with the special method __getitem__.

M[k] = v: Associate value v with key k in map M, replacing the existing value if the map already contains an item with key equal to k. In Python, this is implemented with the special method __setitem__.

del M[k]: Remove from map M the item with key equal to k; if M has no such item, then raise a KeyError. In Python, this is implemented with the special method __delitem__.

len(M): Return the number of items in map M. In Python, this is implemented with the special method __len__.

iter(M): The default iteration for a map generates a sequence of *keys* in the map. In Python, this is implemented with the special method __iter__, and it allows loops of the form, **for** k **in** M.

We have highlighted the above five behaviors because they demonstrate the core functionality of a map—namely, the ability to query, add, modify, or delete a key-value pair, and the ability to report all such pairs. For additional convenience, map *M* should also support the following behaviors:

k in M: Return True if the map contains an item with key k. In Python, this is implemented with the special __contains__ method.

M.get(k, d=None): Return M[k] if key k exists in the map; otherwise return default value d. This provides a form to query M[k] without risk of a KeyError.

M.setdefault(k, d): If key k exists in the map, simply return M[k]; if key k does not exist, set M[k] = d and return that value.

M.pop(k, d=None): Remove the item associated with key k from the map and return its associated value v. If key k is not in the map, return default value d (or raise KeyError if parameter d is None).

M.popitem( ): Remove an arbitrary key-value pair from the map, and return a (k,v) tuple representing the removed pair. If map is empty, raise a KeyError.

M.clear( ): Remove all key-value pairs from the map.

M.keys( ): Return a set-like view of all keys of M.

M.values( ): Return a set-like view of all values of M.

M.items( ): Return a set-like view of (k,v) tuples for all entries of M.

M.update(M2): Assign M[k] = v for every (k,v) pair in map M2.

M == M2: Return True if maps M and M2 have identical key-value associations.

M != M2: Return True if maps M and M2 do not have identical key-value associations.

Example 10.1: *In the following, we show the effect of a series of operations on an initially empty map storing items with integer keys and single-character values. We use the literal syntax for Python's dict class to describe the map contents.*

Operation	Return Value	Map
len(M)	0	{ }
M['K'] = 2	–	{'K': 2}
M['B'] = 4	–	{'K': 2, 'B': 4}
M['U'] = 2	–	{'K': 2, 'B': 4, 'U': 2}
M['V'] = 8	–	{'K': 2, 'B': 4, 'U': 2, 'V': 8}
M['K'] = 9	–	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M['B']	4	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M['X']	KeyError	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M.get('F')	None	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M.get('F', 5)	5	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M.get('K', 5)	9	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
len(M)	4	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
del M['V']	–	{'K': 9, 'B': 4, 'U': 2}
M.pop('K')	9	{'B': 4, 'U': 2}
M.keys( )	'B', 'U'	{'B': 4, 'U': 2}
M.values( )	4, 2	{'B': 4, 'U': 2}
M.items( )	('B', 4), ('U', 2)	{'B': 4, 'U': 2}
M.setdefault('B', 1)	4	{'B': 4, 'U': 2}
M.setdefault('A', 1)	1	{'A': 1, 'B': 4, 'U': 2}
M.popitem( )	('B', 4)	{'A': 1, 'U': 2}

10.1.2 Application: Counting Word Frequencies

As a case study for using a map, consider the problem of counting the number of occurrences of words in a document. This is a standard task when performing a statistical analysis of a document, for example, when categorizing an email or news article. A map is an ideal data structure to use here, for we can use words as keys and word counts as values. We show such an application in Code Fragment 10.1.

We break apart the original document using a combination of file and string methods that results in a loop over a lowercased version of all whitespace separated pieces of the document. We omit all nonalphabetic characters so that parentheses, apostrophes, and other such punctuation are not considered part of a word.

In terms of map operations, we begin with an empty Python dictionary named freq. During the first phase of the algorithm, we execute the command

```
freq[word] = 1 + freq.get(word, 0)
```

for each word occurrence. We use the get method on the right-hand side because the current word might not exist in the dictionary; the default value of 0 is appropriate in that case.

During the second phase of the algorithm, after the full document has been processed, we examine the contents of the frequency map, looping over freq.items( ) to determine which word has the most occurrences.

```
1  freq = { }
2  for piece in open(filename).read( ).lower( ).split( ):
3    # only consider alphabetic characters within this piece
4    word = ''.join(c for c in piece if c.isalpha( ))
5    if word:    # require at least one alphabetic character
6      freq[word] = 1 + freq.get(word, 0)
7
8  max_word = ''
9  max_count = 0
10 for (w,c) in freq.items( ):        # (key, value) tuples represent (word, count)
11   if c > max_count:
12     max_word = w
13     max_count = c
14 print('The most frequent word is', max_word)
15 print('Its number of occurrences is', max_count)
```

Code Fragment 10.1: A program for counting word frequencies in a document, and reporting the most frequent word. We use Python's dict class for the map. We convert the input to lowercase and ignore any nonalphabetic characters.

10.1.3 Python's MutableMapping Abstract Base Class

Section 2.4.3 provides an introduction to the concept of an ***abstract base class*** and the role of such classes in Python's collections module. Methods that are declared to be abstract in such a base class must be implemented by concrete subclasses. However, an abstract base class may provide *concrete* implementation of other methods that depend upon use of the presumed abstract methods. (This is an example of the ***template method design pattern***.)

The collections module provides two abstract base classes that are relevant to our current discussion: the Mapping and MutableMapping classes. The Mapping class includes all nonmutating methods supported by Python's dict class, while the MutableMapping class extends that to include the mutating methods. What we define as the map ADT in Section 10.1.1 is akin to the MutableMapping abstract base class in Python's collections module.

The significance of these abstract base classes is that they provide a framework to assist in creating a user-defined map class. In particular, the MutableMapping class provides *concrete* implementations for all behaviors other than the first five outlined in Section 10.1.1: __getitem__, __setitem__, __delitem__, __len__, and __iter__. As we implement the map abstraction with various data structures, as long as we provide the five core behaviors, we can inherit all other derived behaviors by simply declaring MutableMapping as a parent class.

To better understand the MutableMapping class, we provide a few examples of how concrete behaviors can be derived from the five core abstractions. For example, the __contains__ method, supporting the syntax k in M, could be implemented by making a guarded attempt to retrieve self[k] to determine if the key exists.

```
def __contains__(self, k):
  try:
    self[k]                          # access via __getitem__ (ignore result)
    return True
  except KeyError:
    return False                     # attempt failed
```

A similar approach might be used to provide the logic of the setdefault method.

```
def setdefault(self, k, d):
  try:
    return self[k]                   # if __getitem__ succeeds, return value
  except KeyError:                   # otherwise:
    self[k] = d                      # set default value with __setitem__
    return d                         # and return that newly assigned value
```

We leave as exercises the implementations of the remaining concrete methods of the MutableMapping class.

10.1.4 Our MapBase Class

We will be providing many different implementations of the map ADT, in the remainder of this chapter and next, using a variety of data structures demonstrating a trade-off of advantages and disadvantages. Figure 10.2 provides a preview of those classes.

The MutableMapping abstract base class, from Python's collections module and discussed in the preceding pages, is a valuable tool when implementing a map. However, in the interest of greater code reuse, we define our own MapBase class, which is itself a subclass of the MutableMapping class. Our MapBase class provides additional support for the composition design pattern. This is a technique we introduced when implementing a priority queue (see Section 9.2.1) in order to group a key-value pair as a single instance for internal use.

More formally, our MapBase class is defined in Code Fragment 10.2, extending the existing MutableMapping abstract base class so that we inherit the many useful concrete methods that class provides. We then define a nonpublic nested _Item class, whose instances are able to store both a key and value. This nested class is reasonably similar in design to the _Item class that was defined within our PriorityQueueBase class in Section 9.2.1, except that for a map we provide support for both equality tests and comparisons, both of which rely on the item's key. The notion of equality is necessary for all of our map implementations, as a way to determine whether a key given as a parameter is equivalent to one that is already stored in the map. The notion of comparisons between keys, using the $<$ operator, will become relevant when we later introduce a ***sorted map ADT*** (Section 10.3).

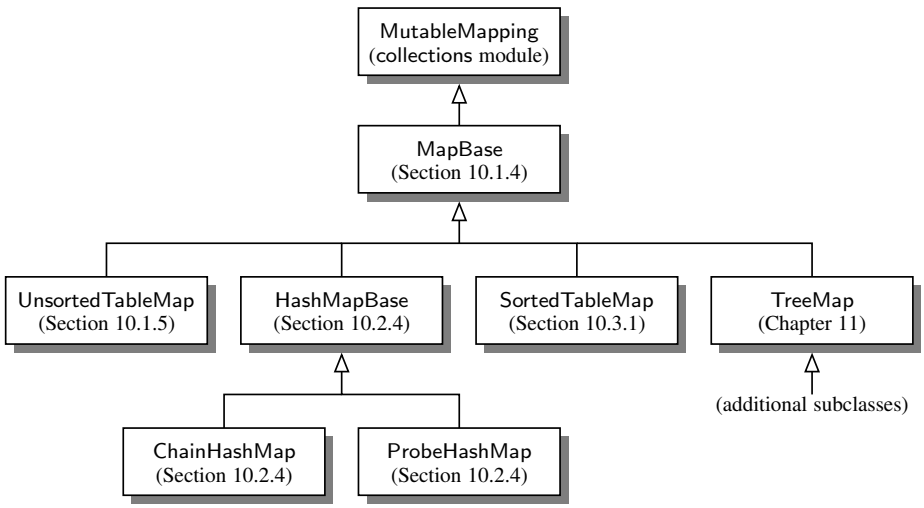


Figure 10.2: Our hierarchy of map types (with references to where they are defined).

```
1  class MapBase(MutableMapping):
2    """Our own abstract base class that includes a nonpublic _Item class."""
3
4    #------------------------------ nested _Item class ------------------------------
5    class _Item:
6      """Lightweight composite to store key-value pairs as map items."""
7      __slots__ = '_key', '_value'
8
9      def __init__(self, k, v):
10       self._key = k
11       self._value = v
12
13     def __eq__(self, other):
14       return self._key == other._key     # compare items based on their keys
15
16     def __ne__(self, other):
17       return not (self == other)         # opposite of __eq__
18
19     def __lt__(self, other):
20       return self._key < other._key      # compare items based on their keys
```

Code Fragment 10.2: Extending the MutableMapping abstract base class to provide a nonpublic _Item class for use in our various map implementations.

10.1.5 Simple Unsorted Map Implementation

We demonstrate the use of the MapBase class with a very simple concrete implementation of the map ADT. Code Fragment 10.3 presents an UnsortedTableMap class that relies on storing key-value pairs in arbitrary order within a Python list.

An empty table is initialized as self._table within the constructor for our map. When a new key is entered into the map, via line 22 of the __setitem__ method, we create a new instance of the nested _Item class, which is inherited from our MapBase class.

This list-based map implementation is simple, but it is not particularly efficient. Each of the fundamental methods, __getitem__, __setitem__, and __delitem__, relies on a for loop to scan the underlying list of items in search of a matching key. In a best-case scenario, such a match may be found near the beginning of the list, in which case the loop terminates; in the worst case, the entire list will be examined. Therefore, each of these methods runs in $O(n)$ time on a map with n items.

```
class UnsortedTableMap(MapBase):
  """Map implementation using an unordered list."""

  def __init__(self):
    """Create an empty map."""
    self._table = [ ]                                   # list of _Item's

  def __getitem__(self, k):
    """Return value associated with key k (raise KeyError if not found)."""
    for item in self._table:
      if k == item._key:
        return item._value
    raise KeyError('Key Error: ' + repr(k))

  def __setitem__(self, k, v):
    """Assign value v to key k, overwriting existing value if present."""
    for item in self._table:
      if k == item._key:                                # Found a match:
        item._value = v                                 # reassign value
        return                                          # and quit
    # did not find match for key
    self._table.append(self._Item(k,v))

  def __delitem__(self, k):
    """Remove item associated with key k (raise KeyError if not found)."""
    for j in range(len(self._table)):
      if k == self._table[j]._key:                      # Found a match:
        self._table.pop(j)                              # remove item
        return                                          # and quit
    raise KeyError('Key Error: ' + repr(k))

  def __len__(self):
    """Return number of items in the map."""
    return len(self._table)

  def __iter__(self):
    """Generate iteration of the map's keys."""
    for item in self._table:
      yield item._key                                   # yield the KEY
```

Code Fragment 10.3: An implementation of a map using a Python list as an unsorted table. Parent class MapBase is given in Code Fragment 10.2.

10.2 Hash Tables

In this section, we introduce one of the most practical data structures for implementing a map, and the one that is used by Python's own implementation of the dict class. This structure is known as a ***hash table***.

Intuitively, a map M supports the abstraction of using keys as indices with a syntax such as M[k]. As a mental warm-up, consider a restricted setting in which a map with n items uses keys that are known to be integers in a range from 0 to $N-1$ for some $N \geq n$. In this case, we can represent the map using a ***lookup table*** of length N, as diagrammed in Figure 10.3.

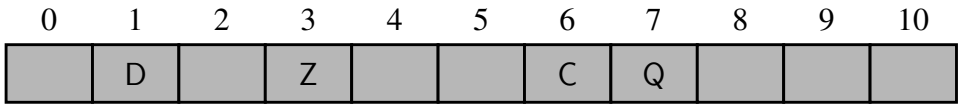


Figure 10.3: A lookup table with length 11 for a map containing items (1,D), (3,Z), (6,C), and (7,Q).

In this representation, we store the value associated with key k at index k of the table (presuming that we have a distinct way to represent an empty slot). Basic map operations of __getitem__, __setitem__, and __delitem__ can be implemented in $O(1)$ worst-case time.

There are two challenges in extending this framework to the more general setting of a map. First, we may not wish to devote an array of length N if it is the case that $N \gg n$. Second, we do not in general require that a map's keys be integers. The novel concept for a hash table is the use of a ***hash function*** to map general keys to corresponding indices in a table. Ideally, keys will be well distributed in the range from 0 to $N-1$ by a hash function, but in practice there may be two or more distinct keys that get mapped to the same index. As a result, we will conceptualize our table as a ***bucket array***, as shown in Figure 10.4, in which each bucket may manage a collection of items that are sent to a specific index by the hash function. (To save space, an empty bucket may be replaced by None.)

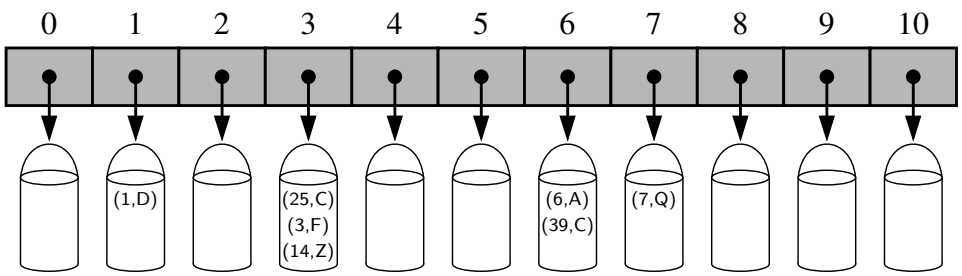


Figure 10.4: A bucket array of capacity 11 with items (1,D), (25,C), (3,F), (14,Z), (6,A), (39,C), and (7,Q), using a simple hash function.

10.2.1 Hash Functions

The goal of a ***hash function***, h, is to map each key k to an integer in the range $[0, N-1]$, where N is the capacity of the bucket array for a hash table. Equipped with such a hash function, h, the main idea of this approach is to use the hash function value, $h(k)$, as an index into our bucket array, A, instead of the key k (which may not be appropriate for direct use as an index). That is, we store the item (k, v) in the bucket $A[h(k)]$.

If there are two or more keys with the same hash value, then two different items will be mapped to the same bucket in A. In this case, we say that a ***collision*** has occurred. To be sure, there are ways of dealing with collisions, which we will discuss later, but the best strategy is to try to avoid them in the first place. We say that a hash function is "good" if it maps the keys in our map so as to sufficiently minimize collisions. For practical reasons, we also would like a hash function to be fast and easy to compute.

It is common to view the evaluation of a hash function, $h(k)$, as consisting of two portions—a ***hash code*** that maps a key k to an integer, and a ***compression function*** that maps the hash code to an integer within a range of indices, $[0, N-1]$, for a bucket array. (See Figure 10.5.)

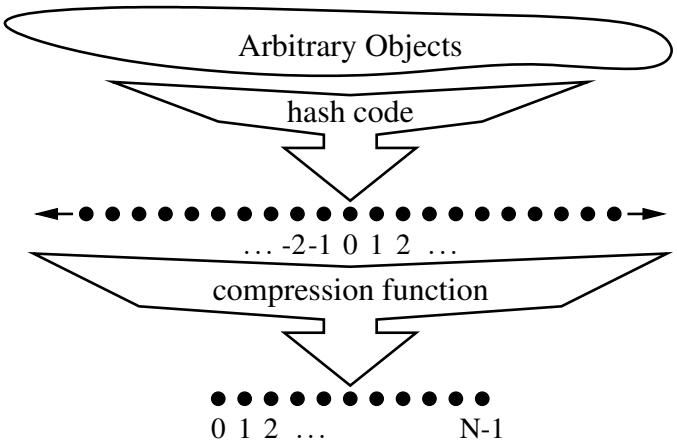


Figure 10.5: Two parts of a hash function: a hash code and a compression function.

The advantage of separating the hash function into two such components is that the hash code portion of that computation is independent of a specific hash table size. This allows the development of a general hash code for each object that can be used for a hash table of any size; only the compression function depends upon the table size. This is particularly convenient, because the underlying bucket array for a hash table may be dynamically resized, depending on the number of items currently stored in the map. (See Section 10.2.3.)

Hash Codes

The first action that a hash function performs is to take an arbitrary key k in our map and compute an integer that is called the ***hash code*** for k; this integer need not be in the range $[0, N-1]$, and may even be negative. We desire that the set of hash codes assigned to our keys should avoid collisions as much as possible. For if the hash codes of our keys cause collisions, then there is no hope for our compression function to avoid them. In this subsection, we begin by discussing the theory of hash codes. Following that, we discuss practical implementations of hash codes in Python.

Treating the Bit Representation as an Integer

To begin, we note that, for any data type X that is represented using at most as many bits as our integer hash codes, we can simply take as a hash code for X an integer interpretation of its bits. For example, the hash code for key 314 could simply be 314. The hash code for a floating-point number such as 3.14 could be based upon an interpretation of the bits of the floating-point representation as an integer.

For a type whose bit representation is longer than a desired hash code, the above scheme is not immediately applicable. For example, Python relies on 32-bit hash codes. If a floating-point number uses a 64-bit representation, its bits cannot be viewed directly as a hash code. One possibility is to use only the high-order 32 bits (or the low-order 32 bits). This hash code, of course, ignores half of the information present in the original key, and if many of the keys in our map only differ in these bits, then they will collide using this simple hash code.

A better approach is to combine in some way the high-order and low-order portions of a 64-bit key to form a 32-bit hash code, which takes all the original bits into consideration. A simple implementation is to add the two components as 32-bit numbers (ignoring overflow), or to take the exclusive-or of the two components. These approaches of combining components can be extended to any object x whose binary representation can be viewed as an n-tuple $(x_0, x_1, \ldots, x_{n-1})$ of 32-bit integers, for example, by forming a hash code for x as $\sum_{i=0}^{n-1} x_i$, or as $x_0 \oplus x_1 \oplus \cdots \oplus x_{n-1}$, where the $\oplus$ symbol represents the bitwise exclusive-or operation (which is ^ in Python).

Polynomial Hash Codes

The summation and exclusive-or hash codes, described above, are not good choices for character strings or other variable-length objects that can be viewed as tuples of the form $(x_0, x_1, \ldots, x_{n-1})$, where the order of the x_i's is significant. For example, consider a 16-bit hash code for a character string s that sums the Unicode values of the characters in s. This hash code unfortunately produces lots of unwanted

collisions for common groups of strings. In particular, "temp01" and "temp10" collide using this function, as do "stop", "tops", "pots", and "spot". A better hash code should somehow take into consideration the positions of the x_i's. An alternative hash code, which does exactly this, is to choose a nonzero constant, $a \neq 1$, and use as a hash code the value

$$x_0 a^{n-1} + x_1 a^{n-2} + \cdots + x_{n-2} a + x_{n-1}.$$

Mathematically speaking, this is simply a polynomial in a that takes the components $(x_0, x_1, \ldots, x_{n-1})$ of an object x as its coefficients. This hash code is therefore called a ***polynomial hash code***. By Horner's rule (see Exercise C-3.50), this polynomial can be computed as

$$x_{n-1} + a(x_{n-2} + a(x_{n-3} + \cdots + a(x_2 + a(x_1 + a x_0)) \cdots)).$$

Intuitively, a polynomial hash code uses multiplication by different powers as a way to spread out the influence of each component across the resulting hash code.

Of course, on a typical computer, evaluating a polynomial will be done using the finite bit representation for a hash code; hence, the value will periodically overflow the bits used for an integer. Since we are more interested in a good spread of the object x with respect to other keys, we simply ignore such overflows. Still, we should be mindful that such overflows are occurring and choose the constant a so that it has some nonzero, low-order bits, which will serve to preserve some of the information content even as we are in an overflow situation.

We have done some experimental studies that suggest that 33, 37, 39, and 41 are particularly good choices for a when working with character strings that are English words. In fact, in a list of over 50,000 English words formed as the union of the word lists provided in two variants of Unix, we found that taking a to be 33, 37, 39, or 41 produced less than 7 collisions in each case!

Cyclic-Shift Hash Codes

A variant of the polynomial hash code replaces multiplication by a with a cyclic shift of a partial sum by a certain number of bits. For example, a 5-bit cyclic shift of the 32-bit value 00111011001011010101000010101000 is achieved by taking the leftmost five bits and placing those on the rightmost side of the representation, resulting in 10110010110101010001010100000111. While this operation has little natural meaning in terms of arithmetic, it accomplishes the goal of varying the bits of the calculation. In Python, a cyclic shift of bits can be accomplished through careful use of the bitwise operators $<<$ and $>>$, taking care to truncate results to 32-bit integers.

An implementation of a cyclic-shift hash code computation for a character string in Python appears as follows:

```
def hash_code(s):
  mask = (1 << 32) - 1                # limit to 32-bit integers
  h = 0
  for character in s:
    h = (h << 5 & mask) | (h >> 27)   # 5-bit cyclic shift of running sum
    h += ord(character)               # add in value of next character
  return h
```

As with the traditional polynomial hash code, fine-tuning is required when using a cyclic-shift hash code, as we must wisely choose the amount to shift by for each new character. Our choice of a 5-bit shift is justified by experiments run on a list of just over 230,000 English words, comparing the number of collisions for various shift amounts (see Table 10.1).

	Collisions	
Shift	**Total**	**Max**
0	234735	623
1	165076	43
2	38471	13
3	7174	5
4	1379	3
5	190	3
6	502	2
7	560	2
8	5546	4
9	393	3
10	5194	5
11	11559	5
12	822	2
13	900	4
14	2001	4
15	19251	8
16	211781	37

Table 10.1: Comparison of collision behavior for the cyclic-shift hash code as applied to a list of 230,000 English words. The "Total" column records the total number of words that collide with at least one other, and the "Max" column records the maximum number of words colliding at any one hash code. Note that with a cyclic shift of 0, this hash code reverts to the one that simply sums all the characters.

Hash Codes in Python

The standard mechanism for computing hash codes in Python is a built-in function with signature hash(x) that returns an integer value that serves as the hash code for object x. However, only ***immutable*** data types are deemed hashable in Python. This restriction is meant to ensure that a particular object's hash code remains constant during that object's lifespan. This is an important property for an object's use as a key in a hash table. A problem could occur if a key were inserted into the hash table, yet a later search were performed for that key based on a different hash code than that which it had when inserted; the wrong bucket would be searched.

Among Python's built-in data types, the immutable int, float, str, tuple, and frozenset classes produce robust hash codes, via the hash function, using techniques similar to those discussed earlier in this section. Hash codes for character strings are well crafted based on a technique similar to polynomial hash codes, except using exclusive-or computations rather than additions. If we repeat the experiment described in Table 10.1 using Python's built-in hash codes, we find that only 8 strings out of the set of more than 230,000 collide with another. Hash codes for tuples are computed with a similar technique based upon a combination of the hash codes of the individual elements of the tuple. When hashing a frozenset, the order of the elements should be irrelevant, and so a natural option is to compute the exclusive-or of the individual hash codes without any shifting. If hash(x) is called for an instance x of a mutable type, such as a list, a TypeError is raised.

Instances of user-defined classes are treated as unhashable by default, with a TypeError raised by the hash function. However, a function that computes hash codes can be implemented in the form of a special method named __hash__ within a class. The returned hash code should reflect the immutable attributes of an instance. It is common to return a hash code that is itself based on the computed hash of the combination of such attributes. For example, a Color class that maintains three numeric red, green, and blue components might implement the method as:

```
def __hash__(self):
  return hash( (self._red, self._green, self._blue) )   # hash combined tuple
```

An important rule to obey is that if a class defines equivalence through __eq__, then any implementation of __hash__ must be consistent, in that if x == y, then hash(x) == hash(y). This is important because if two instances are considered to be equivalent and one is used as a key in a hash table, a search for the second instance should result in the discovery of the first. It is therefore important that the hash code for the second match the hash code for the first, so that the proper bucket is examined. This rule extends to any well-defined comparisons between objects of different classes. For example, since Python treats the expression 5 == 5.0 as true, it ensures that hash(5) and hash(5.0) are the same.

Compression Functions

The hash code for a key k will typically not be suitable for immediate use with a bucket array, because the integer hash code may be negative or may exceed the capacity of the bucket array. Thus, once we have determined an integer hash code for a key object k, there is still the issue of mapping that integer into the range $[0, N-1]$. This computation, known as a ***compression function***, is the second action performed as part of an overall hash function. A good compression function is one that minimizes the number of collisions for a given set of distinct hash codes.

The Division Method

A simple compression function is the ***division method***, which maps an integer i to

$$i \bmod N,$$

where N, the size of the bucket array, is a fixed positive integer. Additionally, if we take N to be a prime number, then this compression function helps "spread out" the distribution of hashed values. Indeed, if N is not prime, then there is greater risk that patterns in the distribution of hash codes will be repeated in the distribution of hash values, thereby causing collisions. For example, if we insert keys with hash codes $\{200, 205, 210, 215, 220, \ldots, 600\}$ into a bucket array of size 100, then each hash code will collide with three others. But if we use a bucket array of size 101, then there will be no collisions. If a hash function is chosen well, it should ensure that the probability of two different keys getting hashed to the same bucket is $1/N$. Choosing N to be a prime number is not always enough, however, for if there is a repeated pattern of hash codes of the form $pN + q$ for several different p's, then there will still be collisions.

The MAD Method

A more sophisticated compression function, which helps eliminate repeated patterns in a set of integer keys, is the ***Multiply-Add-and-Divide*** (or "MAD") method. This method maps an integer i to

$$[(ai + b) \bmod p] \bmod N,$$

where N is the size of the bucket array, p is a prime number larger than N, and a and b are integers chosen at random from the interval $[0, p-1]$, with $a > 0$. This compression function is chosen in order to eliminate repeated patterns in the set of hash codes and get us closer to having a "good" hash function, that is, one such that the probability any two different keys collide is $1/N$. This good behavior would be the same as we would have if these keys were "thrown" into A uniformly at random.

10.2.2 Collision-Handling Schemes

The main idea of a hash table is to take a bucket array, A, and a hash function, h, and use them to implement a map by storing each item (k, v) in the "bucket" $A[h(k)]$. This simple idea is challenged, however, when we have two distinct keys, k_1 and k_2, such that $h(k_1) = h(k_2)$. The existence of such ***collisions*** prevents us from simply inserting a new item (k, v) directly into the bucket $A[h(k)]$. It also complicates our procedure for performing insertion, search, and deletion operations.

Separate Chaining

A simple and efficient way for dealing with collisions is to have each bucket $A[j]$ store its own secondary container, holding items (k, v) such that $h(k) = j$. A natural choice for the secondary container is a small map instance implemented using a list, as described in Section 10.1.5. This ***collision resolution*** rule is known as ***separate chaining***, and is illustrated in Figure 10.6.

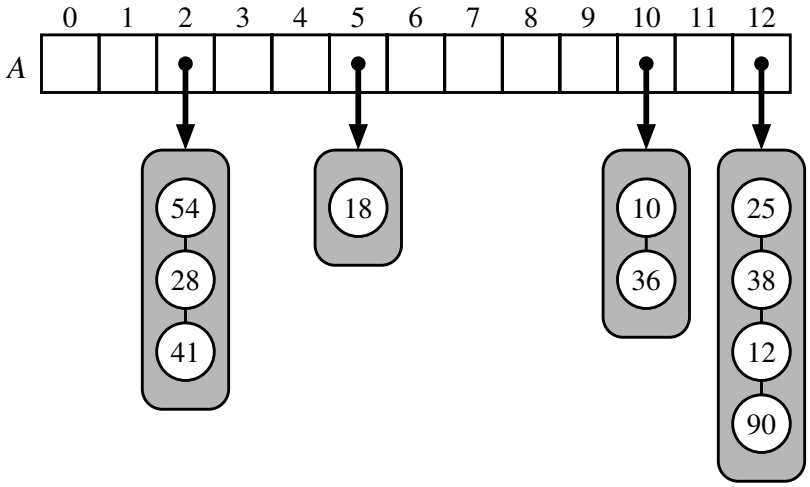


Figure 10.6: A hash table of size 13, storing 10 items with integer keys, with collisions resolved by separate chaining. The compression function is $h(k) = k \bmod 13$. For simplicity, we do not show the values associated with the keys.

In the worst case, operations on an individual bucket take time proportional to the size of the bucket. Assuming we use a good hash function to index the n items of our map in a bucket array of capacity N, the expected size of a bucket is n/N. Therefore, if given a good hash function, the core map operations run in $O(\lceil n/N \rceil)$. The ratio $\lambda = n/N$, called the ***load factor*** of the hash table, should be bounded by a small constant, preferably below 1. As long as λ is $O(1)$, the core operations on the hash table run in $O(1)$ expected time.

Open Addressing

The separate chaining rule has many nice properties, such as affording simple implementations of map operations, but it nevertheless has one slight disadvantage: It requires the use of an auxiliary data structure—a list—to hold items with colliding keys. If space is at a premium (for example, if we are writing a program for a small handheld device), then we can use the alternative approach of always storing each item directly in a table slot. This approach saves space because no auxiliary structures are employed, but it requires a bit more complexity to deal with collisions. There are several variants of this approach, collectively referred to as ***open addressing*** schemes, which we discuss next. Open addressing requires that the load factor is always at most 1 and that items are stored directly in the cells of the bucket array itself.

Linear Probing and Its Variants

A simple method for collision handling with open addressing is ***linear probing***. With this approach, if we try to insert an item (k, v) into a bucket $A[j]$ that is already occupied, where $j = h(k)$, then we next try $A[(j+1) \bmod N]$. If $A[(j+1) \bmod N]$ is also occupied, then we try $A[(j+2) \bmod N]$, and so on, until we find an empty bucket that can accept the new item. Once this bucket is located, we simply insert the item there. Of course, this collision resolution strategy requires that we change the implementation when searching for an existing key—the first step of all `__getitem__`, `__setitem__`, or `__delitem__` operations. In particular, to attempt to locate an item with key equal to k, we must examine consecutive slots, starting from $A[h(k)]$, until we either find an item with that key or we find an empty bucket. (See Figure 10.7.) The name "linear probing" comes from the fact that accessing a cell of the bucket array can be viewed as a "probe."

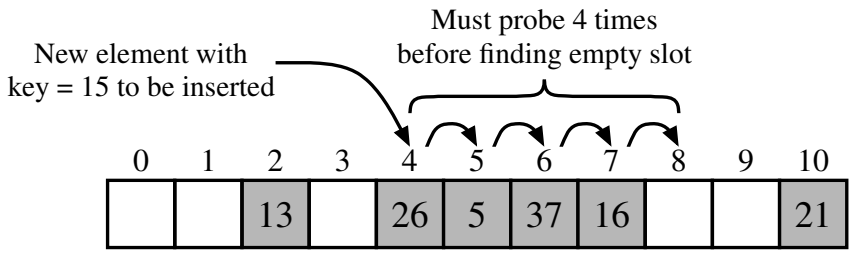


Figure 10.7: Insertion into a hash table with integer keys using linear probing. The hash function is $h(k) = k \bmod 11$. Values associated with keys are not shown.

To implement a deletion, we cannot simply remove a found item from its slot in the array. For example, after the insertion of key 15 portrayed in Figure 10.7, if the item with key 37 were trivially deleted, a subsequent search for 15 would fail because that search would start by probing at index 4, then index 5, and then index 6, at which an empty cell is found. A typical way to get around this difficulty is to replace a deleted item with a special "available" marker object. With this special marker possibly occupying spaces in our hash table, we modify our search algorithm so that the search for a key k will skip over cells containing the available marker and continue probing until reaching the desired item or an empty bucket (or returning back to where we started from). Additionally, our algorithm for `__setitem__` should remember an available cell encountered during the search for k, since this is a valid place to put a new item (k, v), if no existing item is found.

Although use of an open addressing scheme can save space, linear probing suffers from an additional disadvantage. It tends to cluster the items of a map into contiguous runs, which may even overlap (particularly if more than half of the cells in the hash table are occupied). Such contiguous runs of occupied hash cells cause searches to slow down considerably.

Another open addressing strategy, known as ***quadratic probing***, iteratively tries the buckets $A[(h(k) + f(i)) \bmod N]$, for $i = 0, 1, 2, \ldots$, where $f(i) = i^2$, until finding an empty bucket. As with linear probing, the quadratic probing strategy complicates the removal operation, but it does avoid the kinds of clustering patterns that occur with linear probing. Nevertheless, it creates its own kind of clustering, called ***secondary clustering***, where the set of filled array cells still has a non-uniform pattern, even if we assume that the original hash codes are distributed uniformly. When N is prime and the bucket array is less than half full, the quadratic probing strategy is guaranteed to find an empty slot. However, this guarantee is not valid once the table becomes at least half full, or if N is not chosen as a prime number; we explore the cause of this type of clustering in an exercise (C-10.36).

An open addressing strategy that does not cause clustering of the kind produced by linear probing or the kind produced by quadratic probing is the ***double hashing*** strategy. In this approach, we choose a secondary hash function, h', and if h maps some key k to a bucket $A[h(k)]$ that is already occupied, then we iteratively try the buckets $A[(h(k) + f(i)) \bmod N]$ next, for $i = 1, 2, 3, \ldots$, where $f(i) = i \cdot h'(k)$. In this scheme, the secondary hash function is not allowed to evaluate to zero; a common choice is $h'(k) = q - (k \bmod q)$, for some prime number $q < N$. Also, N should be a prime.

Another approach to avoid clustering with open addressing is to iteratively try buckets $A[(h(k) + f(i)) \bmod N]$ where $f(i)$ is based on a pseudo-random number generator, providing a repeatable, but somewhat arbitrary, sequence of subsequent probes that depends upon bits of the original hash code. This is the approach currently used by Python's dictionary class.

10.2.3 Load Factors, Rehashing, and Efficiency

In the hash table schemes described thus far, it is important that the load factor, $\lambda = n/N$, be kept below 1. With separate chaining, as λ gets very close to 1, the probability of a collision greatly increases, which adds overhead to our operations, since we must revert to linear-time list-based methods in buckets that have collisions. Experiments and average-case analyses suggest that we should maintain $\lambda < 0.9$ for hash tables with separate chaining.

With open addressing, on the other hand, as the load factor λ grows beyond 0.5 and starts approaching 1, clusters of entries in the bucket array start to grow as well. These clusters cause the probing strategies to "bounce around" the bucket array for a considerable amount of time before they find an empty slot. In Exercise C-10.36, we explore the degradation of quadratic probing when $\lambda \geq 0.5$. Experiments suggest that we should maintain $\lambda < 0.5$ for an open addressing scheme with linear probing, and perhaps only a bit higher for other open addressing schemes (for example, Python's implementation of open addressing enforces that $\lambda < 2/3$).

If an insertion causes the load factor of a hash table to go above the specified threshold, then it is common to resize the table (to regain the specified load factor) and to reinsert all objects into this new table. Although we need not define a new hash code for each object, we do need to reapply a new compression function that takes into consideration the size of the new table. Each ***rehashing*** will generally scatter the items throughout the new bucket array. When rehashing to a new table, it is a good requirement for the new array's size to be at least double the previous size. Indeed, if we always double the size of the table with each rehashing operation, then we can amortize the cost of rehashing all the entries in the table against the time used to insert them in the first place (as with dynamic arrays; see Section 5.3).

Efficiency of Hash Tables

Although the details of the average-case analysis of hashing are beyond the scope of this book, its probabilistic basis is quite intuitive. If our hash function is good, then we expect the entries to be uniformly distributed in the N cells of the bucket array. Thus, to store n entries, the expected number of keys in a bucket would be $\lceil n/N \rceil$, which is $O(1)$ if n is $O(N)$.

The costs associated with a periodic rehashing, to resize a table after occasional insertions or deletions can be accounted for separately, leading to an additional $O(1)$ amortized cost for `__setitem__` and `__getitem__`.

In the worst case, a poor hash function could map every item to the same bucket. This would result in linear-time performance for the core map operations with separate chaining, or with any open addressing model in which the secondary sequence of probes depends only on the hash code. A summary of these costs is given in Table 10.2.

Operation	List	Hash Table	
		expected	worst case
__getitem__	$O(n)$	$O(1)$	$O(n)$
__setitem__	$O(n)$	$O(1)$	$O(n)$
__delitem__	$O(n)$	$O(1)$	$O(n)$
__len__	$O(1)$	$O(1)$	$O(1)$
__iter__	$O(n)$	$O(n)$	$O(n)$

Table 10.2: Comparison of the running times of the methods of a map realized by means of an unsorted list (as in Section 10.1.5) or a hash table. We let n denote the number of items in the map, and we assume that the bucket array supporting the hash table is maintained such that its capacity is proportional to the number of items in the map.

In practice, hash tables are among the most efficient means for implementing a map, and it is essentially taken for granted by programmers that their core operations run in constant time. Python's dict class is implemented with hashing, and the Python interpreter relies on dictionaries to retrieve an object that is referenced by an identifier in a given namespace. (See Sections 1.10 and 2.5.) The basic command c = a + b involves two calls to __getitem__ in the dictionary for the local namespace to retrieve the values identified as a and b, and a call to __setitem__ to store the result associated with name c in that namespace. In our own algorithm analysis, we simply presume that such dictionary operations run in constant time, independent of the number of entries in the namespace. (Admittedly, the number of entries in a typical namespace can almost surely be bounded by a constant.)

In a 2003 academic paper [31], researchers discuss the possibility of exploiting a hash table's worst-case performance to cause a denial-of-service (DoS) attack of Internet technologies. For many published algorithms that compute hash codes, they note that an attacker could precompute a very large number of moderate-length strings that all hash to the identical 32-bit hash code. (Recall that by any of the hashing schemes we describe, other than double hashing, if two keys are mapped to the same hash code, they will be inseparable in the collision resolution.)

In late 2011, another team of researchers demonstrated an implementation of just such an attack [61]. Web servers allow a series of key-value parameters to be embedded in a URL using a syntax such as `?key1=val1&key2=val2&key3=val3`. Typically, those key-value pairs are immediately stored in a map by the server, and a limit is placed on the length and number of such parameters presuming that storage time in the map will be linear in the number of entries. If all keys were to collide, that storage requires quadratic time (causing the server to perform an inordinate amount of work). In spring of 2012, Python developers distributed a security patch that introduces randomization into the computation of hash codes for strings, making it less tractable to reverse engineer a set of colliding strings.

10.2.4 Python Hash Table Implementation

In this section, we develop two implementations of a hash table, one using separate chaining and the other using open addressing with linear probing. While these approaches to collision resolution are quite different, there are a great many commonalities to the hashing algorithms. For that reason, we extend the MapBase class (from Code Fragment 10.2), to define a new HashMapBase class (see Code Fragment 10.4), providing much of the common functionality to our two hash table implementations. The main design elements of the HashMapBase class are:

- The bucket array is represented as a Python list, named self._table, with all entries initialized to None.
- We maintain an instance variable self._n that represents the number of distinct items that are currently stored in the hash table.
- If the load factor of the table increases beyond 0.5, we double the size of the table and rehash all items into the new table.
- We define a _hash_function utility method that relies on Python's built-in hash function to produce hash codes for keys, and a randomized Multiply-Add-and-Divide (MAD) formula for the compression function.

What is not implemented in the base class is any notion of how a "bucket" should be represented. With separate chaining, each bucket will be an independent structure. With open addressing, however, there is no tangible container for each bucket; the "buckets" are effectively interleaved due to the probing sequences.

In our design, the HashMapBase class presumes the following to be abstract methods, which must be implemented by each concrete subclass:

- _bucket_getitem(j, k)
  This method should search bucket j for an item having key k, returning the associated value, if found, or else raising a KeyError.
- _bucket_setitem(j, k, v)
  This method should modify bucket j so that key k becomes associated with value v. If the key already exists, the new value overwrites the existing value. Otherwise, a new item is inserted and *this method is responsible for incrementing* self._n.
- _bucket_delitem(j, k)
  This method should remove the item from bucket j having key k, or raise a KeyError if no such item exists. (self._n is decremented *after* this method.)
- __iter__
  This is the standard map method to iterate through all keys of the map. Our base class does not delegate this on a per-bucket basis because "buckets" in open addressing are not inherently disjoint.

```
1  class HashMapBase(MapBase):
2    """Abstract base class for map using hash-table with MAD compression."""
3
4    def __init__(self, cap=11, p=109345121):
5      """Create an empty hash-table map."""
6      self._table = cap * [ None ]
7      self._n = 0                                 # number of entries in the map
8      self._prime = p                             # prime for MAD compression
9      self._scale = 1 + randrange(p-1)            # scale from 1 to p-1 for MAD
10     self._shift = randrange(p)                  # shift from 0 to p-1 for MAD
11
12   def _hash_function(self, k):
13     return (hash(k)*self._scale + self._shift) % self._prime % len(self._table)
14
15   def __len__(self):
16     return self._n
17
18   def __getitem__(self, k):
19     j = self._hash_function(k)
20     return self._bucket_getitem(j, k)           # may raise KeyError
21
22   def __setitem__(self, k, v):
23     j = self._hash_function(k)
24     self._bucket_setitem(j, k, v)               # subroutine maintains self._n
25     if self._n > len(self._table) // 2:         # keep load factor <= 0.5
26       self._resize(2 * len(self._table) - 1)    # number 2^x - 1 is often prime
27
28   def __delitem__(self, k):
29     j = self._hash_function(k)
30     self._bucket_delitem(j, k)                  # may raise KeyError
31     self._n -= 1
32
33   def _resize(self, c):                 # resize bucket array to capacity c
34     old = list(self.items())            # use iteration to record existing items
35     self._table = c * [None]            # then reset table to desired capacity
36     self._n = 0                         # n recomputed during subsequent adds
37     for (k,v) in old:
38       self[k] = v                       # reinsert old key-value pair
```

Code Fragment 10.4: A base class for our hash table implementations, extending our MapBase class from Code Fragment 10.2.

Separate Chaining

Code Fragment 10.5 provides a concrete implementation of a hash table with separate chaining, in the form of the ChainHashMap class. To represent a single bucket, it relies on an instance of the UnsortedTableMap class from Code Fragment 10.3.

The first three methods in the class use index j to access the potential bucket in the bucket array, and a check for the special case in which that table entry is None. The only time we need a new bucket structure is when _bucket_setitem is called on an otherwise empty slot. The remaining functionality relies on map behaviors that are already supported by the individual UnsortedTableMap instances. We need a bit of forethought to determine whether the application of __setitem__ on the chain causes a net increase in the size of the map (that is, whether the given key is new).

```
1  class ChainHashMap(HashMapBase):
2    """Hash map implemented with separate chaining for collision resolution."""
3
4    def _bucket_getitem(self, j, k):
5      bucket = self._table[j]
6      if bucket is None:
7        raise KeyError('Key Error: ' + repr(k))          # no match found
8      return bucket[k]                                   # may raise KeyError
9
10   def _bucket_setitem(self, j, k, v):
11     if self._table[j] is None:
12       self._table[j] = UnsortedTableMap( )             # bucket is new to the table
13     oldsize = len(self._table[j])
14     self._table[j][k] = v
15     if len(self._table[j]) > oldsize:                  # key was new to the table
16       self._n += 1                                     # increase overall map size
17
18   def _bucket_delitem(self, j, k):
19     bucket = self._table[j]
20     if bucket is None:
21       raise KeyError('Key Error: ' + repr(k))          # no match found
22     del bucket[k]                                      # may raise KeyError
23
24   def __iter__(self):
25     for bucket in self._table:
26       if bucket is not None:                           # a nonempty slot
27         for key in bucket:
28           yield key
```

Code Fragment 10.5: Concrete hash map class with separate chaining.

Linear Probing

Our implementation of a ProbeHashMap class, using open addressing with linear probing, is given in Code Fragments 10.6 and 10.7. In order to support deletions, we use a technique described in Section 10.2.2 in which we place a special marker in a table location at which an item has been deleted, so that we can distinguish between it and a location that has always been empty. In our implementation, we declare a class-level attribute, _AVAIL, as a sentinel. (We use an instance of the built-in object class because we do not care about any behaviors of the sentinel, just our ability to differentiate it from other objects.)

The most challenging aspect of open addressing is to properly trace the series of probes when collisions occur during an insertion or search for an item. To this end, we define a nonpublic utility, _find_slot, that searches for an item with key k in "bucket" j (that is, where j is the index returned by the hash function for key k).

```
1  class ProbeHashMap(HashMapBase):
2    """Hash map implemented with linear probing for collision resolution."""
3    _AVAIL = object( )       # sentinal marks locations of previous deletions
4
5    def _is_available(self, j):
6      """Return True if index j is available in table."""
7      return self._table[j] is None or self._table[j] is ProbeHashMap._AVAIL
8
9    def _find_slot(self, j, k):
10     """Search for key k in bucket at index j.
11
12     Return (success, index) tuple, described as follows:
13     If match was found, success is True and index denotes its location.
14     If no match found, success is False and index denotes first available slot.
15     """
16     firstAvail = None
17     while True:
18       if self._is_available(j):
19         if firstAvail is None:
20           firstAvail = j                          # mark this as first avail
21         if self._table[j] is None:
22           return (False, firstAvail)              # search has failed
23       elif k == self._table[j]._key:
24         return (True, j)                          # found a match
25       j = (j + 1) % len(self._table)              # keep looking (cyclically)
```

Code Fragment 10.6: Concrete ProbeHashMap class that uses linear probing for collision resolution (continued in Code Fragment 10.7).

```
26   def _bucket_getitem(self, j, k):
27     found, s = self._find_slot(j, k)
28     if not found:
29       raise KeyError('Key Error: ' + repr(k))          # no match found
30     return self._table[s]._value
31
32   def _bucket_setitem(self, j, k, v):
33     found, s = self._find_slot(j, k)
34     if not found:
35       self._table[s] = self._Item(k,v)                 # insert new item
36       self._n += 1                                     # size has increased
37     else:
38       self._table[s]._value = v                        # overwrite existing
39
40   def _bucket_delitem(self, j, k):
41     found, s = self._find_slot(j, k)
42     if not found:
43       raise KeyError('Key Error: ' + repr(k))          # no match found
44     self._table[s] = ProbeHashMap._AVAIL               # mark as vacated
45
46   def __iter__(self):
47     for j in range(len(self._table)):                  # scan entire table
48       if not self._is_available(j):
49         yield self._table[j]._key
```

Code Fragment 10.7: Concrete ProbeHashMap class that uses linear probing for collision resolution (continued from Code Fragment 10.6).

The three primary map operations each rely on the _find_slot utility. When attempting to retrieve the value associated with a given key, we must continue probing until we find the key, or until we reach a table slot with the None value. We cannot stop the search upon reaching an _AVAIL sentinel, because it represents a location that may have been filled when the desired item was once inserted.

When a key-value pair is being assigned in the map, we must attempt to find an existing item with the given key, so that we might overwrite its value, before adding a new item to the map. Therefore, we must search beyond any occurrences of the _AVAIL sentinel when inserting. However, if no match is found, we prefer to repurpose the first slot marked with _AVAIL, if any, when placing the new element in the table. The _find_slot method enacts this logic, continuing the search until a truly empty slot, but returning the index of the first available slot for an insertion.

When deleting an existing item within _bucket_delitem, we intentionally set the table entry to the _AVAIL sentinel in accordance with our strategy.

10.3 Sorted Maps

The traditional map ADT allows a user to look up the value associated with a given key, but the search for that key is a form known as an ***exact search***.

For example, computer systems often maintain information about events that have occurred (such as financial transactions), organizing such events based upon what are known as ***time stamps***. If we can assume that time stamps are unique for a particular system, then we might organize a map with a time stamp serving as the key, and a record about the event that occurred at that time as the value. A particular time stamp could serve as a reference ID for an event, in which case we can quickly retrieve information about that event from the map. However, the map ADT does not provide any way to get a list of all events ordered by the time at which they occur, or to search for which event occurred closest to a particular time. In fact, the fast performance of hash-based implementations of the map ADT relies on the intentionally scattering of keys that may seem very "near" to each other in the original domain, so that they are more uniformly distributed in a hash table.

In this section, we introduce an extension known as the ***sorted map*** ADT that includes all behaviors of the standard map, plus the following:

M.find_min( ): Return the (key,value) pair with minimum key (or None, if map is empty).

M.find_max( ): Return the (key,value) pair with maximum key (or None, if map is empty).

M.find_lt(k): Return the (key,value) pair with the greatest key that is strictly less than k (or None, if no such item exists).

M.find_le(k): Return the (key,value) pair with the greatest key that is less than or equal to k (or None, if no such item exists).

M.find_gt(k): Return the (key,value) pair with the least key that is strictly greater than k (or None, if no such item exists).

M.find_ge(k): Return the (key,value) pair with the least key that is greater than or equal to k (or None, if no such item).

M.find_range(start, stop): Iterate all (key,value) pairs with start $<=$ key $<$ stop. If start is None, iteration begins with minimum key; if stop is None, iteration concludes with maximum key.

iter(M): Iterate all keys of the map according to their natural order, from smallest to largest.

reversed(M): Iterate all keys of the map in reverse order; in Python, this is implemented with the __reversed__ method.

10.3.1 Sorted Search Tables

Several data structures can efficiently support the sorted map ADT, and we will examine some advanced techniques in Section 10.4 and Chapter 11. In this section, we begin by exploring a simple implementation of a sorted map. We store the map's items in an array-based sequence A so that they are in increasing order of their keys, assuming the keys have a naturally defined order. (See Figure 10.8.) We refer to this implementation of a map as a ***sorted search table***.

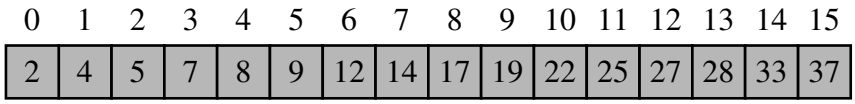


Figure 10.8: Realization of a map by means of a sorted search table. We show only the keys for this map, so as to highlight their ordering.

As was the case with the unsorted table map of Section 10.1.5, the sorted search table has a space requirement that is $O(n)$, assuming we grow and shrink the array to keep its size proportional to the number of items in the map. The primary advantage of this representation, and our reason for insisting that A be array-based, is that it allows us to use the ***binary search*** algorithm for a variety of efficient operations.

Binary Search and Inexact Searches

We originally presented the binary search algorithm in Section 4.1.3, as a means for detecting whether a given target is stored within a sorted sequence. In our original presentation (Code Fragment 4.3 on page 156), a binary_search function returned True of False to designate whether the desired target was found. While such an approach could be used to implement the __contains__ method of the map ADT, we can adapt the binary search algorithm to provide far more useful information when performing forms of inexact search in support of the sorted map ADT.

The important realization is that while performing a binary search, we can determine the index at or near where a target might be found. During a successful search, the standard implementation determines the precise index at which the target is found. During an unsuccessful search, although the target is not found, the algorithm will effectively determine a pair of indices designating elements of the collection that are just less than or just greater than the missing target.

As a motivating example, our original simulation from Figure 4.5 on page 156 shows a successful binary search for a target of 22, using the same data we portray in Figure 10.8. Had we instead been searching for 21, the first four steps of the algorithm would be the same. The subsequent difference is that we would make an additional call with inverted parameters high=9 and low=10, effectively concluding that the missing target lies in the gap between values 19 and 22 in that example.

Implementation

In Code Fragments 10.8 through 10.10, we present a complete implementation of a class, SortedTableMap, that supports the sorted map ADT. The most notable feature of our design is the inclusion of a _find_index utility function. This method using the binary search algorithm, but by convention returns the *index* of the leftmost item in the search interval having key greater than or equal to k. Therefore, if the key is present, it will return the index of the item having that key. (Recall that keys are unique in a map.) When the key is missing, the function returns the index of the item in the search interval that is just beyond where the key would have been located. As a technicality, the method returns index high $+ 1$ to indicate that no items of the interval had a key greater than k.

We rely on this utility method when implementing the traditional map operations and the new sorted map operations. The body of each of the __getitem__, __setitem__, and __delitem__ methods begins with a call to _find_index to determine a candidate index at which a matching key might be found. For __getitem__, we simply check whether that is a valid index containing the target to determine the result. For __setitem__, recall that the goal is to replace the value of an existing item, if one with key k is found, but otherwise to insert a new item into the map. The index returned by _find_index will be the index of the match, if one exists, or otherwise the exact index at which the new item should be inserted. For __delitem__, we again rely on the convenience of _find_index to determine the location of the item to be popped, if any.

Our _find_index utility is equally valuable when implementing the various inexact search methods given in Code Fragment 10.10. For each of the methods find_lt, find_le, find_gt, and find_ge, we begin with a call to _find_index utility, which locates the first index at which there is an element with key $\geq k$, if any. This is precisely what we want for find_ge, if valid, and just beyond the index we want for find_lt. For find_gt and find_le we need some extra case analysis to distinguish whether the indicated index has a key equal to k. For example, if the indicated item has a matching key, our find_gt implementation increments the index before continuing with the process. (We omit the implementation of find_le, for brevity.) In all cases, we must properly handle boundary cases, reporting None when unable to find a key with the desired property.

Our strategy for implementing find_range is to use the _find_index utility to locate the first item with key $\geq$ start (assuming start is not None). With that knowledge, we use a while loop to sequentially report items until reaching one that has a key greater than or equal to the stopping value (or until reaching the end of the table). It is worth noting that the while loop may trivially iterate zero items if the first key that is greater than or equal to start also happens to be greater than or equal to stop. This represents an empty range in the map.

```
1  class SortedTableMap(MapBase):
2    """Map implementation using a sorted table."""
3
4    #---------------------------- nonpublic behaviors ----------------------------
5    def _find_index(self, k, low, high):
6      """Return index of the leftmost item with key greater than or equal to k.
7
8      Return high + 1 if no such item qualifies.
9
10     That is, j will be returned such that:
11       all items of slice table[low:j] have key < k
12       all items of slice table[j:high+1] have key >= k
13     """
14     if high < low:
15       return high + 1                                  # no element qualifies
16     else:
17       mid = (low + high) // 2
18       if k == self._table[mid]._key:
19         return mid                                     # found exact match
20       elif k < self._table[mid]._key:
21         return self._find_index(k, low, mid - 1)       # Note: may return mid
22       else:
23         return self._find_index(k, mid + 1, high)      # answer is right of mid
24
25   #---------------------------- public behaviors ----------------------------
26   def __init__(self):
27     """Create an empty map."""
28     self._table = [ ]
29
30   def __len__(self):
31     """Return number of items in the map."""
32     return len(self._table)
33
34   def __getitem__(self, k):
35     """Return value associated with key k (raise KeyError if not found)."""
36     j = self._find_index(k, 0, len(self._table) - 1)
37     if j == len(self._table) or self._table[j]._key != k:
38       raise KeyError('Key Error: ' + repr(k))
39     return self._table[j]._value
```

Code Fragment 10.8: An implementation of a SortedTableMap class (continued in Code Fragments 10.9 and 10.10).

```
40    def __setitem__(self, k, v):
41      """Assign value v to key k, overwriting existing value if present."""
42      j = self._find_index(k, 0, len(self._table) - 1)
43      if j < len(self._table) and self._table[j]._key == k:
44        self._table[j]._value = v                          # reassign value
45      else:
46        self._table.insert(j, self._Item(k,v))             # adds new item
47
48    def __delitem__(self, k):
49      """Remove item associated with key k (raise KeyError if not found)."""
50      j = self._find_index(k, 0, len(self._table) - 1)
51      if j == len(self._table) or self._table[j]._key != k:
52        raise KeyError('Key Error: ' + repr(k))
53      self._table.pop(j)                                   # delete item
54
55    def __iter__(self):
56      """Generate keys of the map ordered from minimum to maximum."""
57      for item in self._table:
58        yield item._key
59
60    def __reversed__(self):
61      """Generate keys of the map ordered from maximum to minimum."""
62      for item in reversed(self._table):
63        yield item._key
64
65    def find_min(self):
66      """Return (key,value) pair with minimum key (or None if empty)."""
67      if len(self._table) > 0:
68        return (self._table[0]._key, self._table[0]._value)
69      else:
70        return None
71
72    def find_max(self):
73      """Return (key,value) pair with maximum key (or None if empty)."""
74      if len(self._table) > 0:
75        return (self._table[-1]._key, self._table[-1]._value)
76      else:
77        return None
```

Code Fragment 10.9: An implementation of a SortedTableMap class (together with Code Fragments 10.8 and 10.10).

```
78    def find_ge(self, k):
79      """Return (key,value) pair with least key greater than or equal to k."""
80      j = self._find_index(k, 0, len(self._table) - 1)             # j's key >= k
81      if j < len(self._table):
82        return (self._table[j]._key, self._table[j]._value)
83      else:
84        return None
85
86    def find_lt(self, k):
87      """Return (key,value) pair with greatest key strictly less than k."""
88      j = self._find_index(k, 0, len(self._table) - 1)             # j's key >= k
89      if j > 0:
90        return (self._table[j-1]._key, self._table[j-1]._value) # Note use of j-1
91      else:
92        return None
93
94    def find_gt(self, k):
95      """Return (key,value) pair with least key strictly greater than k."""
96      j = self._find_index(k, 0, len(self._table) - 1)             # j's key >= k
97      if j < len(self._table) and self._table[j]._key == k:
98        j += 1                                             # advanced past match
99      if j < len(self._table):
100       return (self._table[j]._key, self._table[j]._value)
101     else:
102       return None
103
104   def find_range(self, start, stop):
105     """Iterate all (key,value) pairs such that start <= key < stop.
106
107     If start is None, iteration begins with minimum key of map.
108     If stop is None, iteration continues through the maximum key of map.
109     """
110     if start is None:
111       j = 0
112     else:
113       j = self._find_index(start, 0, len(self._table)-1)       # find first result
114     while j < len(self._table) and (stop is None or self._table[j]._key < stop):
115       yield (self._table[j]._key, self._table[j]._value)
116       j += 1
```

Code Fragment 10.10: An implementation of a SortedTableMap class (continued from Code Fragments 10.9 and 10.10). We omit the find_le method due to space.

Analysis

We conclude by analyzing the performance of our SortedTableMap implementation. A summary of the running times for all methods of the sorted map ADT (including the traditional map operations) is given in Table 10.3. It should be clear that the __len__, find_min, and find_max methods run in $O(1)$ time, and that iterating the keys of the table in either direction can be peformed in $O(n)$ time.

The analysis for the various forms of search all depend on the fact that a binary search on a table with n entries runs in $O(\log n)$ time. This claim was originally shown as Proposition 4.2 in Section 4.2, and that analysis clearly applies to our _find_index method as well. We therefore claim an $O(\log n)$ worst-case running time for methods __getitem__, find_lt, find_gt, find_le, and find_ge. Each of these makes a single call to _find_index, followed by a constant number of additional steps to determine the appropriate answer based on the index. The analysis of find_range is a bit more interesting. It begins with a binary search to find the first item within the range (if any). After that, it executes a loop that takes $O(1)$ time per iteration to report subsequent values until reaching the end of the range. If there are s items reported in the range, the total running time is $O(s + \log n)$.

In contrast to the efficient search operations, update operations for a sorted table may take considerable time. Although binary search can help identify the index at which an update occurs, both insertions and deletions require, in the worst case, that linearly many existing elements be shifted in order to maintain the sorted order of the table. Specifically, the potential call to _table.insert from within __setitem__ and _table.pop from within __delitem__ lead to $O(n)$ worst-case time. (See the discussion of corresponding operations of the list class in Section 5.4.1.)

In conclusion, sorted tables are primarily used in situations where we expect many searches but relatively few updates.

Operation	Running Time
len(M)	$O(1)$
k in M	$O(\log n)$
M[k] = v	$O(n)$ worst case; $O(\log n)$ if existing k
del M[k]	$O(n)$ worst case
M.find_min( ), M.find_max( )	$O(1)$
M.find_lt(k), M.find_gt(k) M.find_le(k), M.find_ge(k)	$O(\log n)$
M.find_range(start, stop)	$O(s + \log n)$ where s items are reported
iter(M), reversed(M)	$O(n)$

Table 10.3: Performance of a sorted map, as implemented with SortedTableMap. We use n to denote the number of items in the map at the time the operation is performed. The space requirement is $O(n)$.

10.3.2 Two Applications of Sorted Maps

In this section, we explore applications in which there is particular advantage to using a *sorted* map rather than a traditional (unsorted) map. To apply a sorted map, keys must come from a domain that is totally ordered. Furthermore, to take advantage of the inexact or range searches afforded by a sorted map, there should be some reason why nearby keys have relevance to a search.

Flight Databases

There are several Web sites on the Internet that allow users to perform queries on flight databases to find flights between various cities, typically with the intent to buy a ticket. To make a query, a user specifies origin and destination cities, a departure date, and a departure time. To support such queries, we can model the flight database as a map, where keys are Flight objects that contain fields corresponding to these four parameters. That is, a key is a tuple

$$k = (\mathsf{origin}, \mathsf{destination}, \mathsf{date}, \mathsf{time}).$$

Additional information about a flight, such as the flight number, the number of seats still available in first (F) and coach (Y) class, the flight duration, and the fare, can be stored in the value object.

Finding a requested flight is not simply a matter of finding an exact match for a requested query. Although a user typically wants to exactly match the origin and destination cities, he or she may have flexibility for the departure date, and certainly will have some flexibility for the departure time on a specific day. We can handle such a query by ordering our keys lexicographically. Then, an efficient implementation for a sorted map would be a good way to satisfy users' queries. For instance, given a user query key k, we could call find_ge(k) to return the first flight between the desired cities, having a departure date and time matching the desired query or later. Better yet, with well-constructed keys, we could use find_range(k1, k2) to find all flights within a given range of times. For example, if $k1 = (\mathsf{ORD}, \mathsf{PVD}, \mathsf{05May}, \mathsf{09{:}30})$, and $k2 = (\mathsf{ORD}, \mathsf{PVD}, \mathsf{05May}, \mathsf{20{:}00})$, a respective call to find_range(k1, k2) might result in the following sequence of key-value pairs:

(ORD, PVD, 05May, 09:53) : (AA 1840, F5, Y15, 02:05, $251),
(ORD, PVD, 05May, 13:29) : (AA 600, F2, Y0, 02:16, $713),
(ORD, PVD, 05May, 17:39) : (AA 416, F3, Y9, 02:09, $365),
(ORD, PVD, 05May, 19:50) : (AA 1828, F9, Y25, 02:13, $186)

Maxima Sets

Life is full of trade-offs. We often have to trade off a desired performance measure against a corresponding cost. Suppose, for the sake of an example, we are interested in maintaining a database rating automobiles by their maximum speeds and their cost. We would like to allow someone with a certain amount of money to query our database to find the fastest car they can possibly afford.

We can model such a trade-off problem as this by using a key-value pair to model the two parameters that we are trading off, which in this case would be the pair $(\mathsf{cost}, \mathsf{speed})$ for each car. Notice that some cars are strictly better than other cars using this measure. For example, a car with cost-speed pair $(20000, 100)$ is strictly better than a car with cost-speed pair $(30000, 90)$. At the same time, there are some cars that are not strictly dominated by another car. For example, a car with cost-speed pair $(20000, 100)$ may be better or worse than a car with cost-speed pair $(30000, 120)$, depending on how much money we have to spend. (See Figure 10.9.)

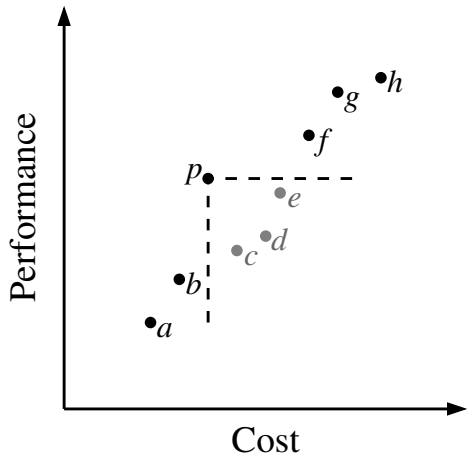


Figure 10.9: Illustrating the cost-performance trade-off with pairs represented by points in the plane. Notice that point p is strictly better than points c, d, and e, but may be better or worse than points a, b, f, g, and h, depending on the price we are willing to pay. Thus, if we were to add p to our set, we could remove the points c, d, and e, but not the others.

Formally, we say a cost-performance pair (a,b) ***dominates*** pair $(c,d) \neq (a,b)$ if $a \leq c$ and $b \geq d$, that is, if the first pair has no greater cost and at least as good performance. A pair (a,b) is called a ***maximum*** pair if it is not dominated by any other pair. We are interested in maintaining the set of maxima of a collection of cost-performance pairs. That is, we would like to add new pairs to this collection (for example, when a new car is introduced), and to query this collection for a given dollar amount, d, to find the fastest car that costs no more than d dollars.

Maintaining a Maxima Set with a Sorted Map

We can store the set of maxima pairs in a sorted map, M, so that the cost is the key field and performance (speed) is the value field. We can then implement operations add(c,p), which adds a new cost-performance pair (c,p), and best(c), which returns the best pair with cost at most c, as shown in Code Fragment 10.11.

```
1  class CostPerformanceDatabase:
2    """Maintain a database of maximal (cost,performance) pairs."""
3
4    def __init__(self):
5      """Create an empty database."""
6      self._M = SortedTableMap( )              # or a more efficient sorted map
7
8    def best(self, c):
9      """Return (cost,performance) pair with largest cost not exceeding c.
10
11     Return None if there is no such pair.
12     """
13     return self._M.find_le(c)
14
15   def add(self, c, p):
16     """Add new entry with cost c and performance p."""
17     # determine if (c,p) is dominated by an existing pair
18     other = self._M.find_le(c)                # other is at least as cheap as c
19     if other is not None and other[1] >= p: # if its performance is as good,
20         return                                # (c,p) is dominated, so ignore
21     self._M[c] = p                            # else, add (c,p) to database
22     # and now remove any pairs that are dominated by (c,p)
23     other = self._M.find_gt(c)                # other more expensive than c
24     while other is not None and other[1] <= p:
25       del self._M[other[0]]
26       other = self._M.find_gt(c)
```

Code Fragment 10.11: An implementation of a class maintaining a set of maxima cost-performance pairs using a sorted map.

Unfortunately, if we implement M using the SortedTableMap, the add behavior has $O(n)$ worst-case running time. If, on the other hand, we implement M using a skip list, which we next describe, we can perform best(c) queries in $O(\log n)$ expected time and add(c,p) updates in $O((1+r)\log n)$ expected time, where r is the number of points removed.

10.4 Skip Lists

An interesting data structure for realizing the sorted map ADT is the ***skip list***. In Section 10.3.1, we saw that a sorted array will allow $O(\log n)$-time searches via the binary search algorithm. Unfortunately, update operations on a sorted array have $O(n)$ worst-case running time because of the need to shift elements. In Chapter 7 we demonstrated that linked lists support very efficient update operations, as long as the position within the list is identified. Unfortunately, we cannot perform fast searches on a standard linked list; for example, the binary search algorithm requires an efficient means for direct accessing an element of a sequence by index.

Skip lists provide a clever compromise to efficiently support search and update operations. A ***skip list*** S for a map M consists of a series of lists $\{S_0, S_1, \ldots, S_h\}$. Each list S_i stores a subset of the items of M sorted by increasing keys, plus items with two sentinel keys denoted $-\infty$ and $+\infty$, where $-\infty$ is smaller than every possible key that can be inserted in M and $+\infty$ is larger than every possible key that can be inserted in M. In addition, the lists in S satisfy the following:

- List S_0 contains every item of the map M (plus sentinels $-\infty$ and $+\infty$).
- For $i = 1, \ldots, h-1$, list S_i contains (in addition to $-\infty$ and $+\infty$) a randomly generated subset of the items in list S_{i-1}.
- List S_h contains only $-\infty$ and $+\infty$.

An example of a skip list is shown in Figure 10.10. It is customary to visualize a skip list S with list S_0 at the bottom and lists $S_1, \ldots, S_h$ above it. Also, we refer to h as the ***height*** of skip list S.

Intuitively, the lists are set up so that S_{i+1} contains more or less alternate items of S_i. As we shall see in the details of the insertion method, the items in S_{i+1} are chosen at random from the items in S_i by picking each item from S_i to also be in S_{i+1} with probability $1/2$. That is, in essence, we "flip a coin" for each item in S_i

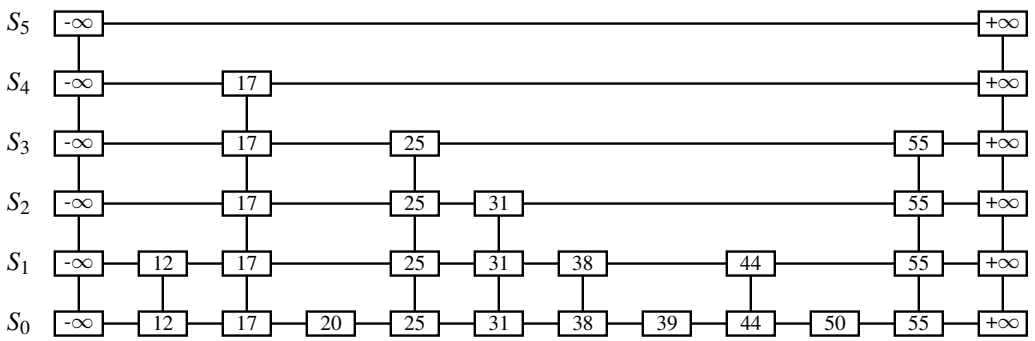


Figure 10.10: Example of a skip list storing 10 items. For simplicity, we show only the items' keys, not their associated values.

and place that item in S_{i+1} if the coin comes up "heads." Thus, we expect S_1 to have about $n/2$ items, S_2 to have about $n/4$ items, and, in general, S_i to have about $n/2^i$ items. In other words, we expect the height h of S to be about $\log n$. The halving of the number of items from one list to the next is not enforced as an explicit property of skip lists, however. Instead, randomization is used.

Functions that generate numbers that can be viewed as random numbers are built into most modern computers, because they are used extensively in computer games, cryptography, and computer simulations, Some functions, called ***pseudo-random number generators***, generate random-like numbers, starting with an initial ***seed***. (See discusion of random module in Section 1.11.1.) Other methods use hardware devices to extract "true" random numbers from nature. In any case, we will assume that our computer has access to numbers that are sufficiently random for our analysis.

The main advantage of using ***randomization*** in data structure and algorithm design is that the structures and functions that result are usually simple and efficient. The skip list has the same logarithmic time bounds for searching as is achieved by the binary search algorithm, yet it extends that performance to update methods when inserting or deleting items. Nevertheless, the bounds are ***expected*** for the skip list, while binary search has a ***worst-case*** bound with a sorted table.

A skip list makes random choices in arranging its structure in such a way that search and update times are $O(\log n)$ ***on average***, where n is the number of items in the map. Interestingly, the notion of average time complexity used here does not depend on the probability distribution of the keys in the input. Instead, it depends on the use of a random-number generator in the implementation of the insertions to help decide where to place the new item. The running time is averaged over all possible outcomes of the random numbers used when inserting entries.

Using the position abstraction used for lists and trees, we view a skip list as a two-dimensional collection of positions arranged horizontally into ***levels*** and vertically into ***towers***. Each level is a list S_i and each tower contains positions storing the same item across consecutive lists. The positions in a skip list can be traversed using the following operations:

next(p): Return the position following p on the same level.

prev(p): Return the position preceding p on the same level.

below(p): Return the position below p in the same tower.

above(p): Return the position above p in the same tower.

We conventionally assume that the above operations return None if the position requested does not exist. Without going into the details, we note that we can easily implement a skip list by means of a linked structure such that the individual traversal methods each take $O(1)$ time, given a skip-list position p. Such a linked structure is essentially a collection of h doubly linked lists aligned at towers, which are also doubly linked lists.

10.4.1 Search and Update Operations in a Skip List

The skip-list structure affords simple map search and update algorithms. In fact, all of the skip-list search and update algorithms are based on an elegant SkipSearch method that takes a key k and finds the position p of the item in list S_0 that has the largest key less than or equal to k (which is possibly $-\infty$).

Searching in a Skip List

Suppose we are given a search key k. We begin the SkipSearch method by setting a position variable p to the topmost, left position in the skip list S, called the ***start position*** of S. That is, the start position is the position of S_h storing the special entry with key $-\infty$. We then perform the following steps (see Figure 10.11), where $\mathsf{key}(p)$ denotes the key of the item at position p:

1. If $S.\mathsf{below}(p)$ is None, then the search terminates—we are ***at the bottom*** and have located the item in S with the largest key less than or equal to the search key k. Otherwise, we ***drop down*** to the next lower level in the present tower by setting $p = S.\mathsf{below}(p)$.
2. Starting at position p, we move p forward until it is at the rightmost position on the present level such that $\mathsf{key}(p) \leq k$. We call this the ***scan forward*** step. Note that such a position always exists, since each level contains the keys $+\infty$ and $-\infty$. It may be that p remains where it started after we perform such a forward scan for this level.
3. Return to step 1.

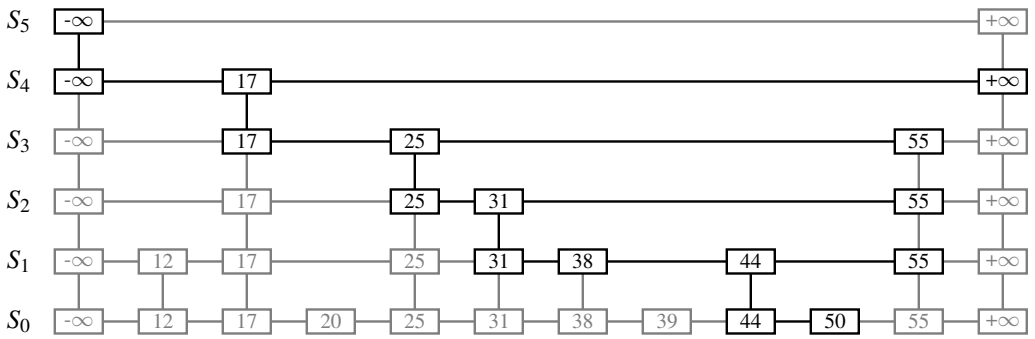


Figure 10.11: Example of a search in a skip list. The positions examined when searching for key 50 are highlighted.

We give a pseudo-code description of the skip-list search algorithm, SkipSearch, in Code Fragment 10.12. Given this method, the map operation $M[k]$ is performed by computing $p = \mathsf{SkipSearch}(k)$ and testing whether or not $\mathsf{key}(p) = k$. If these two keys are equal, we return the associated value; otherwise, we raise a KeyError.

Algorithm SkipSearch(k):
Input: A search key k
Output: Position p in the bottom list S_0 with the largest key such that key(p) $\leq$ k

```
p = start                                   {begin at start position}
while below(p) ≠ None do
  p = below(p)                              {drop down}
  while k ≥ key(next(p)) do
    p = next(p)                             {scan forward}
return p.
```

Code Fragment 10.12: Algorithm to search a skip list S for key k.

As it turns out, the expected running time of algorithm SkipSearch on a skip list with n entries is $O(\log n)$. We postpone the justification of this fact, however, until after we discuss the implementation of the update methods for skip lists. Navigation starting at the position identified by SkipSearch(k) can be easily used to provide the additional forms of searches in the sorted map ADT (e.g., find_gt, find_range).

Insertion in a Skip List

The execution of the map operation M[k] = v begins with a call to SkipSearch(k). This gives us the position p of the bottom-level item with the largest key less than or equal to k (note that p may hold the special item with key $-\infty$). If key(p) $= k$, the associated value is overwritten with v. Otherwise, we need to create a new tower for item (k,v). We insert (k,v) immediately after position p within S_0. After inserting the new item at the bottom level, we use randomization to decide the height of the tower for the new item. We "flip" a coin, and if the flip comes up tails, then we stop here. Else (the flip comes up heads), we backtrack to the previous (next higher) level and insert (k,v) in this level at the appropriate position. We again flip a coin; if it comes up heads, we go to the next higher level and repeat. Thus, we continue to insert the new item (k,v) in lists until we finally get a flip that comes up tails. We link together all the references to the new item (k,v) created in this process to create its tower. A coin flip can be simulated with Python's built-in pseudo-random number generator from the random module by calling randrange(2), which returns 0 or 1, each with probability $1/2$.

We give the insertion algorithm for a skip list S in Code Fragment 10.13 and we illustrate it in Figure 10.12. The algorithm uses an insertAfterAbove($p,q,(k,v)$) method that inserts a position storing the item (k,v) after position p (on the same level as p) and above position q, returning the new position r (and setting internal references so that next, prev, above, and below methods will work correctly for p, q, and r). The expected running time of the insertion algorithm on a skip list with n entries is $O(\log n)$, which we show in Section 10.4.2.

Algorithm SkipInsert(k,v):
Input: Key k and value v
Output: Topmost position of the item inserted in the skip list

```
p = SkipSearch(k)
q = None                          {q will represent top node in new item's tower}
i = −1
repeat
  i = i + 1
  if i ≥ h then
    h = h + 1                                  {add a new level to the skip list}
    t = next(s)
    s = insertAfterAbove(None, s, (−∞, None))      {grow leftmost tower}
    insertAfterAbove(s, t, (+∞, None))             {grow rightmost tower}
  while above(p) is None do
    p = prev(p)                                            {scan backward}
  p = above(p)                                        {jump up to higher level}
  q = insertAfterAbove(p, q, (k, v))    {increase height of new item's tower}
until coinFlip() == tails
n = n + 1
return q
```

Code Fragment 10.13: Insertion in a skip list. Method coinFlip() returns "heads" or "tails", each with probability $1/2$. Instance variables n, h, and s hold the number of entries, the height, and the start node of the skip list.

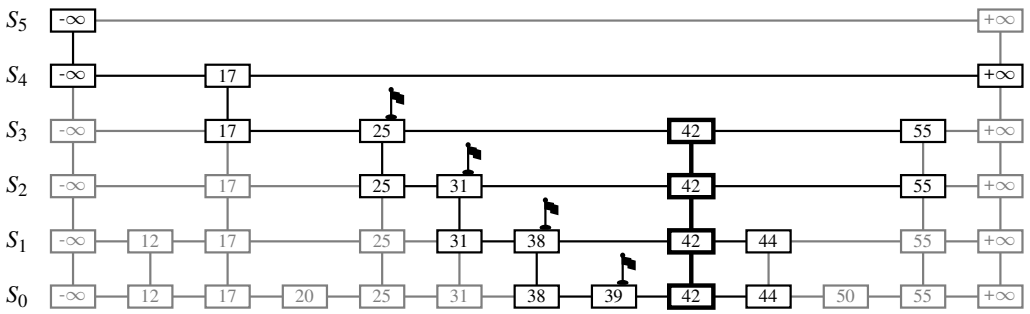


Figure 10.12: Insertion of an entry with key 42 into the skip list of Figure 10.10. We assume that the random "coin flips" for the new entry came up heads three times in a row, followed by tails. The positions visited are highlighted. The positions inserted to hold the new entry are drawn with thick lines, and the positions preceding them are flagged.

Removal in a Skip List

Like the search and insertion algorithms, the removal algorithm for a skip list is quite simple. In fact, it is even easier than the insertion algorithm. That is, to perform the map operation del M[k] we begin by executing method SkipSearch(k). If the position p stores an entry with key different from k, we raise a KeyError. Otherwise, we remove p and all the positions above p, which are easily accessed by using above operations to climb up the tower of this entry in S starting at position p. While removing levels of the tower, we reestablish links between the horizontal neighbors of each removed position. The removal algorithm is illustrated in Figure 10.13 and a detailed description of it is left as an exercise (R-10.24). As we show in the next subsection, deletion operation in a skip list with n entries has $O(\log n)$ expected running time.

Before we give this analysis, however, there are some minor improvements to the skip-list data structure we would like to discuss. First, we do not actually need to store references to values at the levels of the skip list above the bottom level, because all that is needed at these levels are references to keys. In fact, we can more efficiently represent a tower as a single object, storing the key-value pair, and maintaining j previous references and j next references if the tower reaches level S_j. Second, for the horizontal axes, it is possible to keep the list singly linked, storing only the next references. We can perform insertions and removals in strictly a top-down, scan-forward fashion. We explore the details of this optimization in Exercise C-10.44. Neither of these optimizations improve the asymptotic performance of skip lists by more than a constant factor, but these improvements can, nevertheless, be meaningful in practice. In fact, experimental evidence suggests that optimized skip lists are faster in practice than AVL trees and other balanced search trees, which are discussed in Chapter 11.

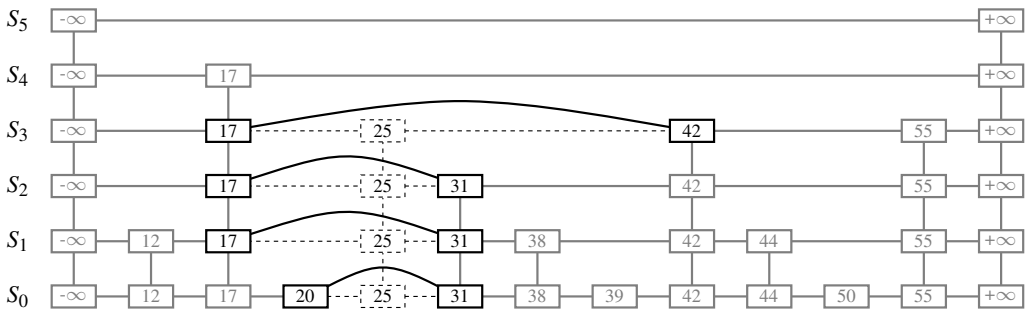


Figure 10.13: Removal of the entry with key 25 from the skip list of Figure 10.12. The positions visited after the search for the position of S_0 holding the entry are highlighted. The positions removed are drawn with dashed lines.

Maintaining the Topmost Level

A skip list S must maintain a reference to the start position (the topmost, left position in S) as an instance variable, and must have a policy for any insertion that wishes to continue inserting a new entry past the top level of S. There are two possible courses of action we can take, both of which have their merits.

One possibility is to restrict the top level, h, to be kept at some fixed value that is a function of n, the number of entries currently in the map (from the analysis we will see that $h = \max\{10, 2\lceil \log n \rceil\}$ is a reasonable choice, and picking $h = 3\lceil \log n \rceil$ is even safer). Implementing this choice means that we must modify the insertion algorithm to stop inserting a new position once we reach the topmost level (unless $\lceil \log n \rceil < \lceil \log(n+1) \rceil$, in which case we can now go at least one more level, since the bound on the height is increasing).

The other possibility is to let an insertion continue inserting a new position as long as heads keeps getting returned from the random number generator. This is the approach taken by algorithm SkipInsert of Code Fragment 10.13. As we show in the analysis of skip lists, the probability that an insertion will go to a level that is more than $O(\log n)$ is very low, so this design choice should also work.

Either choice will still result in the expected $O(\log n)$ time to perform search, insertion, and removal, however, which we show in the next section.

10.4.2 Probabilistic Analysis of Skip Lists ⋆

As we have shown above, skip lists provide a simple implementation of a sorted map. In terms of worst-case performance, however, skip lists are not a superior data structure. In fact, if we do not officially prevent an insertion from continuing significantly past the current highest level, then the insertion algorithm can go into what is almost an infinite loop (it is not actually an infinite loop, however, since the probability of having a fair coin repeatedly come up heads forever is 0). Moreover, we cannot infinitely add positions to a list without eventually running out of memory. In any case, if we terminate position insertion at the highest level h, then the ***worst-case*** running time for performing the __getitem__, __setitem__, and __delitem__ map operations in a skip list S with n entries and height h is $O(n+h)$. This worst-case performance occurs when the tower of every entry reaches level $h-1$, where h is the height of S. However, this event has very low probability. Judging from this worst case, we might conclude that the skip-list structure is strictly inferior to the other map implementations discussed earlier in this chapter. But this would not be a fair analysis, for this worst-case behavior is a gross overestimate.

Bounding the Height of a Skip List

Because the insertion step involves randomization, a more accurate analysis of skip lists involves a bit of probability. At first, this might seem like a major undertaking, for a complete and thorough probabilistic analysis could require deep mathematics (and, indeed, there are several such deep analyses that have appeared in data structures research literature). Fortunately, such an analysis is not necessary to understand the expected asymptotic behavior of skip lists. The informal and intuitive probabilistic analysis we give below uses only basic concepts of probability theory.

Let us begin by determining the expected value of the height h of a skip list S with n entries (assuming that we do not terminate insertions early). The probability that a given entry has a tower of height $i \geq 1$ is equal to the probability of getting i consecutive heads when flipping a coin, that is, this probability is $1/2^i$. Hence, the probability P_i that level i has at least one position is at most

$$P_i \leq \frac{n}{2^i},$$

for the probability that any one of n different events occurs is at most the sum of the probabilities that each occurs.

The probability that the height h of S is larger than i is equal to the probability that level i has at least one position, that is, it is no more than P_i. This means that h is larger than, say, $3\log n$ with probability at most

$$\begin{aligned} P_{3\log n} &\leq \frac{n}{2^{3\log n}} \\ &= \frac{n}{n^3} = \frac{1}{n^2}. \end{aligned}$$

For example, if $n = 1000$, this probability is a one-in-a-million long shot. More generally, given a constant $c > 1$, h is larger than $c\log n$ with probability at most $1/n^{c-1}$. That is, the probability that h is smaller than $c\log n$ is at least $1 - 1/n^{c-1}$. Thus, with high probability, the height h of S is $O(\log n)$.

Analyzing Search Time in a Skip List

Next, consider the running time of a search in skip list S, and recall that such a search involves two nested **while** loops. The inner loop performs a scan forward on a level of S as long as the next key is no greater than the search key k, and the outer loop drops down to the next level and repeats the scan forward iteration. Since the height h of S is $O(\log n)$ with high probability, the number of drop-down steps is $O(\log n)$ with high probability.

So we have yet to bound the number of scan-forward steps we make. Let n_i be the number of keys examined while scanning forward at level i. Observe that, after the key at the starting position, each additional key examined in a scan-forward at level i cannot also belong to level $i+1$. If any of these keys were on the previous level, we would have encountered them in the previous scan-forward step. Thus, the probability that any key is counted in n_i is $1/2$. Therefore, the expected value of n_i is exactly equal to the expected number of times we must flip a fair coin before it comes up heads. This expected value is 2. Hence, the expected amount of time spent scanning forward at any level i is $O(1)$. Since S has $O(\log n)$ levels with high probability, a search in S takes expected time $O(\log n)$. By a similar analysis, we can show that the expected running time of an insertion or a removal is $O(\log n)$.

Space Usage in a Skip List

Finally, let us turn to the space requirement of a skip list S with n entries. As we observed above, the expected number of positions at level i is $n/2^i$, which means that the expected total number of positions in S is

$$\sum_{i=0}^{h} \frac{n}{2^i} = n \sum_{i=0}^{h} \frac{1}{2^i}.$$

Using Proposition 3.5 on geometric summations, we have

$$\sum_{i=0}^{h} \frac{1}{2^i} = \frac{\left(\frac{1}{2}\right)^{h+1} - 1}{\frac{1}{2} - 1} = 2 \cdot \left(1 - \frac{1}{2^{h+1}}\right) < 2 \quad \text{for all } h \geq 0.$$

Hence, the expected space requirement of S is $O(n)$.

Table 10.4 summarizes the performance of a sorted map realized by a skip list.

Operation	Running Time
len(M)	$O(1)$
k **in** M	$O(\log n)$ expected
M[k] = v	$O(\log n)$ expected
del M[k]	$O(\log n)$ expected
M.find_min( ), M.find_max( )	$O(1)$
M.find_lt(k), M.find_gt(k) M.find_le(k), M.find_ge(k)	$O(\log n)$ expected
M.find_range(start, stop)	$O(s + \log n)$ expected, with s items reported
iter(M), reversed(M)	$O(n)$

Table 10.4: Performance of a sorted map implemented with a skip list. We use n to denote the number of entries in the dictionary at the time the operation is performed. The expected space requirement is $O(n)$.

10.5 Sets, Multisets, and Multimaps

We conclude this chapter by examining several additional abstractions that are closely related to the map ADT, and that can be implemented using data structures similar to those for a map.

- A ***set*** is an unordered collection of elements, without duplicates, that typically supports efficient membership tests. In essence, elements of a set are like keys of a map, but without any auxiliary values.
- A ***multiset*** (also known as a ***bag***) is a set-like container that allows duplicates.
- A ***multimap*** is similar to a traditional map, in that it associates values with keys; however, in a multimap the same key can be mapped to multiple values. For example, the index of this book maps a given term to one or more locations at which the term occurs elsewhere in the book.

10.5.1 The Set ADT

Python provides support for representing the mathematical notion of a set through the built-in classes **frozenset** and **set**, as originally discussed in Chapter 1, with frozenset being an immutable form. Both of those classes are implemented using hash tables in Python.

Python's collections module defines abstract base classes that essentially mirror these built-in classes. Although the choice of names is counterintuitive, the abstract base class collections.Set matches the concrete frozenset class, while the abstract base class collections.MutableSet is akin to the concrete set class.

In our own discussion, we equate the "set ADT" with the behavior of the built-in set class (and thus, the collections.MutableSet base class). We begin by listing what we consider to be the five most fundamental behaviors for a set S:

S.add(e): Add element e to the set. This has no effect if the set already contains e.

S.discard(e): Remove element e from the set, if present. This has no effect if the set does not contain e.

e in S: Return True if the set contains element e. In Python, this is implemented with the special __contains__ method.

len(S): Return the number of elements in set S. In Python, this is implemented with the special method __len__.

iter(S): Generate an iteration of all elements of the set. In Python, this is implemented with the special method __iter__.

In the next section, we will see that the above five methods suffice for deriving all other behaviors of a set. Those remaining behaviors can be naturally grouped as follows. We begin by describing the following additional operations for removing one or more elements from a set:

S.remove(e): Remove element e from the set. If the set does not contain e, raise a KeyError.

S.pop( ): Remove and return an arbitrary element from the set. If the set is empty, raise a KeyError.

S.clear( ): Remove all elements from the set.

The next group of behaviors perform Boolean comparisons between two sets.

S == T: Return True if sets S and T have identical contents.

S != T: Return True if sets S and T are not equivalent.

S <= T: Return True if set S is a subset of set T.

S < T: Return True if set S is a *proper* subset of set T.

S >= T: Return True if set S is a superset of set T.

S > T: Return True if set S is a *proper* superset of set T.

S.isdisjoint(T): Return True if sets S and T have no common elements.

Finally, there exists a variety of behaviors that either update an existing set, or compute a new set instance, based on classical set theory operations.

S | T: Return a new set representing the union of sets S and T.

S |= T: Update set S to be the union of S and set T.

S & T: Return a new set representing the intersection of sets S and T.

S &= T: Update set S to be the intersection of S and set T.

S ^ T: Return a new set representing the symmetric difference of sets S and T, that is, a set of elements that are in precisely one of S or T.

S ^= T: Update set S to become the symmetric difference of itself and set T.

S − T: Return a new set containing elements in S but not T.

S −= T: Update set S to remove all common elements with set T.

10.5.2 Python's MutableSet Abstract Base Class

To aid in the creation of user-defined set classes, Python's collections module provides a MutableSet abstract base class (just as it provides the MutableMapping abstract base class discussed in Section 10.1.3). The MutableSet base class provides concrete implementations for all methods described in Section 10.5.1, except for five core behaviors (add, discard, __contains__, __len__, and __iter__) that must be implemented by any concrete subclass. This design is an example of what is known as the ***template method pattern***, as the concrete methods of the MutableSet class rely on the presumed abstract methods that will subsequently be provided by a subclass.

For the purpose of illustration, we examine algorithms for implementing several of the derived methods of the MutableSet base class. For example, to determine if one set is a proper subset of another, we must verify two conditions: a proper subset must have size strictly smaller than that of its superset, and each element of a subset must be contained in the superset. An implementation of the corresponding __lt__ method based on this logic is given in Code Fragment 10.14.

```
def __lt__(self, other):       # supports syntax S < T
  """Return true if this set is a proper subset of other."""
  if len(self) >= len(other):
    return False               # proper subset must have strictly smaller size
  for e in self:
    if e not in other:
      return False             # not a subset since element missing from other
  return True                  # success; all conditions are met
```

Code Fragment 10.14: A possible implementation of the MutableSet.__lt__ method, which tests if one set is a proper subset of another.

As another example, we consider the computation of the union of two sets. The set ADT includes two forms for computing a union. The syntax S | T should produce a new set that has contents equal to the union of existing sets S and T. This operation is implemented through the special method __or__ in Python. Another syntax, S |= T is used to *update* existing set S to become the union of itself and set T. Therefore, all elements of T that are not already contained in S should be added to S. We note that this "in-place" operation may be implemented more efficiently than if we were to rely on the first form, using the syntax S = S | T, in which identifier S is reassigned to a new set instance that represents the union. For convenience, Python's built-in set class supports named version of these behaviors, with S.union(T) equivalent to S | T, and S.update(T) equivalent to S |= T (yet, those named versions are not formally provided by the MutableSet abstract base class).

```
def __or__(self, other):                    # supports syntax S | T
  """Return a new set that is the union of two existing sets."""
  result = type(self)( )                    # create new instance of concrete class
  for e in self:
    result.add(e)
  for e in other:
    result.add(e)
  return result
```

Code Fragment 10.15: An implementation of the MutableSet.__or__ method, which computes the union of two existing sets.

An implementation of the behavior that computes a new set as a union of two others is given in the form of the __or__ special method, in Code Fragment 10.15. An important subtlety in this implementation is the instantiation of the resulting set. Since the MutableSet class is designed as an abstract base class, instances must belong to a concrete subclass. When computing the union of two such concrete instances, the result should presumably be an instance of the same class as the operands. The function type(self) returns a reference to the actual class of the instance identified as self, and the subsequent parentheses in expression type(self)( ) call the default constructor for that class.

In terms of efficiency, we analyze such set operations while letting n denote the size of S and m denote the size of set T for an operation such as S | T. If the concrete sets are implemented with hashing, the expected running time of the implementation in Code Fragment 10.15 is $O(m+n)$, because it loops over both sets, performing constant-time operations in the form of a containment check and a possible insertion into the result.

Our implementation of the in-place version of a union is given in Code Fragment 10.16, in the form of the __ior__ special method that supports syntax S |= T. Notice that in this case, we do not create a new set instance, instead we modify and return the existing set, after updating its contents to reflect the union operation. The in-place version of the union has expected running time $O(m)$ where m is the size of the second set, because we only have to loop through that second set.

```
def __ior__(self, other):                   # supports syntax S |= T
  """Modify this set to be the union of itself an another set."""
  for e in other:
    self.add(e)
  return self                               # technical requirement of in-place operator
```

Code Fragment 10.16: An implementation of the MutableSet.__ior__ method, which performs an in-place union of one set with another.

10.5.3 Implementing Sets, Multisets, and Multimaps

Sets

Although sets and maps have very different public interfaces, they are really quite similar. A set is simply a map in which keys do not have associated values. Any data structure used to implement a map can be modified to implement the set ADT with similar performance guarantees. We could trivially adapt any map class by storing set elements as keys, and using None as an irrelevant value, but such an implementation is unnecessarily wasteful. An efficient set implementation should abandon the _Item composite that we use in our MapBase class and instead store set elements directly in a data structure.

Multisets

The same element may occur several times in a multiset. All of the data structures we have seen can be reimplemented to allow for duplicates to appear as separate elements. However, another way to implement a multiset is by using a *map* in which the map key is a (distinct) element of the multiset, and the associated value is a count of the number of occurrences of that element within the multiset. In fact, that is essentially what we did in Section 10.1.2 when computing the frequency of words within a document.

Python's standard collections module includes a definition for a class named Counter that is in essence a multiset. Formally, the Counter class is a subclass of dict, with the expectation that values are integers, and with additional functionality like a most_common(n) method that returns a list of the n most common elements. The standard __iter__ reports each element only once (since those are formally the keys of the dictionary). There is another method named elements( ) that iterates through the multiset with each element being repeated according to its count.

Multimaps

Although there is no multimap in Python's standard libraries, a common implementation approach is to use a standard map in which the value associated with a key is itself a container class storing any number of associated values. We give an example of such a MultiMap class in Code Fragment 10.17. Our implementation uses the standard dict class as the map, and a list of values as a composite value in the dictionary. We have designed the class so that a different map implementation can easily be substituted by overriding the class-level _MapType attribute at line 3.

```
class MultiMap:
  """A multimap class built upon use of an underlying map for storage."""
  _MapType = dict                    # Map type; can be redefined by subclass

  def __init__(self):
    """Create a new empty multimap instance."""
    self._map = self._MapType( )          # create map instance for storage
    self._n = 0

  def __iter__(self):
    """Iterate through all (k,v) pairs in multimap."""
    for k,secondary in self._map.items( ):
      for v in secondary:
        yield (k,v)

  def add(self, k, v):
    """Add pair (k,v) to multimap."""
    container = self._map.setdefault(k, [ ])      # create empty list, if needed
    container.append(v)
    self._n += 1

  def pop(self, k):
    """Remove and return arbitrary (k,v) with key k (or raise KeyError)."""
    secondary = self._map[k]                      # may raise KeyError
    v = secondary.pop( )
    if len(secondary) == 0:
      del self._map[k]                            # no pairs left
    self._n -= 1
    return (k, v)

  def find(self, k):
    """Return arbitrary (k,v) pair with given key (or raise KeyError)."""
    secondary = self._map[k]                      # may raise KeyError
    return (k, secondary[0])

  def find_all(self, k):
    """Generate iteration of all (k,v) pairs with given key."""
    secondary = self._map.get(k, [ ])             # empty list, by default
    for v in secondary:
      yield (k,v)
```

Code Fragment 10.17: An implementation of a MultiMap using a dict for storage. The __len__ method, which returns self._n, is omitted from this listing.

10.6 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-10.1 Give a concrete implementation of the pop method in the context of the MutableMapping class, relying only on the five primary abstract methods of that class.

R-10.2 Give a concrete implementation of the items( ) method in the context of the MutableMapping class, relying only on the five primary abstract methods of that class. What would its running time be if directly applied to the UnsortedTableMap subclass?

R-10.3 Give a concrete implementation of the items( ) method directly within the UnsortedTableMap class, ensuring that the entire iteration runs in $O(n)$ time.

R-10.4 What is the worst-case running time for inserting n key-value pairs into an initially empty map M that is implemented with the UnsortedTableMap class?

R-10.5 Reimplement the UnsortedTableMap class from Section 10.1.5, using the PositionalList class from Section 7.4 rather than a Python list.

R-10.6 Which of the hash table collision-handling schemes could tolerate a load factor above 1 and which could not?

R-10.7 Our Position classes for lists and trees support the __eq__ method so that two distinct position instances are considered equivalent if they refer to the same underlying node in a structure. For positions to be allowed as keys in a hash table, there must be a definition for the __hash__ method that is consistent with this notion of equivalence. Provide such a __hash__ method.

R-10.8 What would be a good hash code for a vehicle identification number that is a string of numbers and letters of the form "9X9XX99X9XX999999," where a "9" represents a digit and an "X" represents a letter?

R-10.9 Draw the 11-entry hash table that results from using the hash function, $h(i) = (3i+5) \bmod 11$, to hash the keys 12, 44, 13, 88, 23, 94, 11, 39, 20, 16, and 5, assuming collisions are handled by chaining.

R-10.10 What is the result of the previous exercise, assuming collisions are handled by linear probing?

R-10.11 Show the result of Exercise R-10.9, assuming collisions are handled by quadratic probing, up to the point where the method fails.

R-10.12 What is the result of Exercise R-10.9 when collisions are handled by double hashing using the secondary hash function $h'(k) = 7 - (k \bmod 7)$?

R-10.13 What is the worst-case time for putting n entries in an initially empty hash table, with collisions resolved by chaining? What is the best case?

R-10.14 Show the result of rehashing the hash table shown in Figure 10.6 into a table of size 19 using the new hash function $h(k) = 3k \bmod 17$.

R-10.15 Our HashMapBase class maintains a load factor $\lambda \leq 0.5$. Reimplement that class to allow the user to specify the maximum load, and adjust the concrete subclasses accordingly.

R-10.16 Give a pseudo-code description of an insertion into a hash table that uses quadratic probing to resolve collisions, assuming we also use the trick of replacing deleted entries with a special "deactivated entry" object.

R-10.17 Modify our ProbeHashMap to use quadratic probing.

R-10.18 Explain why a hash table is not suited to implement a sorted map.

R-10.19 Describe how a sorted list implemented as a doubly linked list could be used to implement the sorted map ADT.

R-10.20 What is the worst-case asymptotic running time for performing n deletions from a SortedTableMap instance that initially contains $2n$ entries?

R-10.21 Consider the following variant of the _find_index method from Code Fragment 10.8, in the context of the SortedTableMap class:

```
def _find_index(self, k, low, high):
  if high < low:
    return high + 1
  else:
    mid = (low + high) // 2
    if self._table[mid]._key < k:
      return self._find_index(k, mid + 1, high)
    else:
      return self._find_index(k, low, mid - 1)
```

Does this always produce the same result as the original version? Justify your answer.

R-10.22 What is the expected running time of the methods for maintaining a maxima set if we insert n pairs such that each pair has lower cost and performance than one before it? What is contained in the sorted map at the end of this series of operations? What if each pair had a lower cost and higher performance than the one before it?

R-10.23 Draw an example skip list S that results from performing the following series of operations on the skip list shown in Figure 10.13: del S[38], S[48] = 'x', S[24] = 'y', del S[55]. Record your coin flips, as well.

R-10.24 Give a pseudo-code description of the __delitem__ map operation when using a skip list.

R-10.25 Give a concrete implementation of the pop method, in the context of a MutableSet abstract base class, that relies only on the five core set behaviors described in Section 10.5.2.

R-10.26 Give a concrete implementation of the isdisjoint method in the context of the MutableSet abstract base class, relying only on the five primary abstract methods of that class. Your algorithm should run in $O(\min(n,m))$ where n and m denote the respective cardinalities of the two sets.

R-10.27 What abstraction would you use to manage a database of friends' birthdays in order to support efficient queries such as "find all friends whose birthday is today" and "find the friend who will be the next to celebrate a birthday"?

Creativity

C-10.28 On page 406 of Section 10.1.3, we give an implementation of the method setdefault as it might appear in the MutableMapping abstract base class. While that method accomplishes the goal in a general fashion, its efficiency is less than ideal. In particular, when the key is new, there will be a failed search due to the initial use of __getitem__, and then a subsequent insertion via __setitem__. For a concrete implementation, such as the UnsortedTableMap, this is twice the work because a complete scan of the table will take place during the failed __getitem__, and then another complete scan of the table takes place due to the implementation of __setitem__. A better solution is for the UnsortedTableMap class to override setdefault to provide a direct solution that performs a single search. Give such an implementation of UnsortedTableMap.setdefault.

C-10.29 Repeat Exercise C-10.28 for the ProbeHashMap class.

C-10.30 Repeat Exercise C-10.28 for the ChainHashMap class.

C-10.31 For an ideal compression function, the capacity of the bucket array for a hash table should be a prime number. Therefore, we consider the problem of locating a prime number in a range $[M, 2M]$. Implement a method for finding such a prime by using the ***sieve algorithm***. In this algorithm, we allocate a $2M$ cell Boolean array A, such that cell i is associated with the integer i. We then initialize the array cells to all be "true" and we "mark off" all the cells that are multiples of 2, 3, 5, 7, and so on. This process can stop after it reaches a number larger than $\sqrt{2M}$. (Hint: Consider a bootstrapping method for finding the primes up to $\sqrt{2M}$.)

C-10.32 Perform experiments on our ChainHashMap and ProbeHashMap classes to measure its efficiency using random key sets and varying limits on the load factor (see Exercise R-10.15).

C-10.33 Our implementation of separate chaining in ChainHashMap conserves memory by representing empty buckets in the table as None, rather than as empty instances of a secondary structure. Because many of these buckets will hold a single item, a better optimization is to have those slots of the table directly reference the _Item instance, and to reserve use of secondary containers for buckets that have two or more items. Modify our implementation to provide this additional optimization.

C-10.34 Computing a hash code can be expensive, especially for lengthy keys. In our hash table implementations, we compute the hash code when first inserting an item, and recompute each item's hash code each time we resize our table. Python's dict class makes an interesting trade-off. The hash code is computed once, when an item is inserted, and the hash code is stored as an extra field of the item composite, so that it need not be recomputed. Reimplement our HashTableBase class to use such an approach.

C-10.35 Describe how to perform a removal from a hash table that uses linear probing to resolve collisions where we do not use a special marker to represent deleted elements. That is, we must rearrange the contents so that it appears that the removed entry was never inserted in the first place.

C-10.36 The quadratic probing strategy has a clustering problem related to the way it looks for open slots. Namely, when a collision occurs at bucket $h(k)$, it checks buckets $A[(h(k)+i^2) \bmod N]$, for $i = 1,2,\ldots,N-1$.

a. Show that $i^2 \bmod N$ will assume at most $(N+1)/2$ distinct values, for N prime, as i ranges from 1 to $N-1$. As a part of this justification, note that $i^2 \bmod N = (N-i)^2 \bmod N$ for all i.

b. A better strategy is to choose a prime N such that $N \bmod 4 = 3$ and then to check the buckets $A[(h(k) \pm i^2) \bmod N]$ as i ranges from 1 to $(N-1)/2$, alternating between plus and minus. Show that this alternate version is guaranteed to check every bucket in A.

C-10.37 Refactor our ProbeHashMap design so that the sequence of secondary probes for collision resolution can be more easily customized. Demonstrate your new framework by providing separate concrete subclasses for linear probing and quadratic probing.

C-10.38 Design a variation of binary search for performing the multimap operation find_all(k) implemented with a sorted search table that includes duplicates, and show that it runs in time $O(s+\log n)$, where n is the number of elements in the dictionary and s is the number of items with given key k.

C-10.39 Although keys in a map are distinct, the binary search algorithm can be applied in a more general setting in which an array stores possibly duplicative elements in nondecreasing order. Consider the goal of identifying the index of the *leftmost* element with key greater than or equal to given k. Does the _find_index method as given in Code Fragment 10.8 guarantee such a result? Does the _find_index method as given in Exercise R-10.21 guarantee such a result? Justify your answers.

C-10.40 Suppose we are given two sorted search tables S and T, each with n entries (with S and T being implemented with arrays). Describe an $O(\log^2 n)$-time algorithm for finding the kth smallest key in the union of the keys from S and T (assuming no duplicates).

C-10.41 Give an $O(\log n)$-time solution for the previous problem.

C-10.42 Suppose that each row of an $n \times n$ array A consists of 1's and 0's such that, in any row of A, all the 1's come before any 0's in that row. Assuming A is already in memory, describe a method running in $O(n \log n)$ time (not $O(n^2)$ time!) for counting the number of 1's in A.

C-10.43 Given a collection C of n cost-performance pairs (c, p), describe an algorithm for finding the maxima pairs of C in $O(n \log n)$ time.

C-10.44 Show that the methods above(p) and prev(p) are not actually needed to efficiently implement a map using a skip list. That is, we can implement insertions and deletions in a skip list using a strictly top-down, scan-forward approach, without ever using the above or prev methods. (Hint: In the insertion algorithm, first repeatedly flip the coin to determine the level where you should start inserting the new entry.)

C-10.45 Describe how to modify a skip-list representation so that index-based operations, such as retrieving the item at index j, can be performed in $O(\log n)$ expected time.

C-10.46 For sets S and T, the syntax S ^ T returns a new set that is the symmetric difference, that is, a set of elements that are in precisely one of S or T. This syntax is supported by the special __xor__ method. Provide an implementation of that method in the context of the MutableSet abstract base class, relying only on the five primary abstract methods of that class.

C-10.47 In the context of the MutableSet abstract base class, describe a concrete implementation of the __and__ method, which supports the syntax S & T for computing the intersection of two existing sets.

C-10.48 An ***inverted file*** is a critical data structure for implementing a search engine or the index of a book. Given a document D, which can be viewed as an unordered, numbered list of words, an inverted file is an ordered list of words, L, such that, for each word w in L, we store the indices of the places in D where w appears. Design an efficient algorithm for constructing L from D.

C-10.49 Python's collections module provides an OrderedDict class that is unrelated to our sorted map abstraction. An OrderedDict is a subclass of the standard hash-based dict class that retains the expected $O(1)$ performance for the primary map operations, but that also guarantees that the `__iter__` method reports items of the map according to first-in, first-out (FIFO) order. That is, the key that has been in the dictionary the longest is reported first. (The order is unaffected when the value for an existing key is overwritten.) Describe an algorithmic approach for achieving such performance.

Projects

P-10.50 Perform a comparative analysis that studies the collision rates for various hash codes for character strings, such as various polynomial hash codes for different values of the parameter a. Use a hash table to determine collisions, but only count collisions where different strings map to the same hash code (not if they map to the same location in this hash table). Test these hash codes on text files found on the Internet.

P-10.51 Perform a comparative analysis as in the previous exercise, but for 10-digit telephone numbers instead of character strings.

P-10.52 Implement an OrderedDict class, as described in Exercise C-10.49, ensuring that the primary map operations run in $O(1)$ expected time.

P-10.53 Design a Python class that implements the skip-list data structure. Use this class to create a complete implementation of the sorted map ADT.

P-10.54 Extend the previous project by providing a graphical animation of the skip-list operations. Visualize how entries move up the skip list during insertions and are linked out of the skip list during removals. Also, in a search operation, visualize the scan-forward and drop-down actions.

P-10.55 Write a spell-checker class that stores a lexicon of words, W, in a Python set, and implements a method, check(s), which performs a ***spell check*** on the string s with respect to the set of words, W. If s is in W, then the call to check(s) returns a list containing only s, as it is assumed to be spelled correctly in this case. If s is not in W, then the call to check(s) returns a list of every word in W that might be a correct spelling of s. Your program should be able to handle all the common ways that s might be a misspelling of a word in W, including swapping adjacent characters in a word, inserting a single character in between two adjacent characters in a word, deleting a single character from a word, and replacing a character in a word with another character. For an extra challenge, consider phonetic substitutions as well.

Chapter Notes

Hashing is a well-studied technique. The reader interested in further study is encouraged to explore the book by Knuth [65], as well as the book by Vitter and Chen [100]. Skip lists were introduced by Pugh [86]. Our analysis of skip lists is a simplification of a presentation given by Motwani and Raghavan [80]. For a more in-depth analysis of skip lists, please see the various research papers on skip lists that have appeared in the data structures literature [59, 81, 84]. Exercise C-10.36 was contributed by James Lee.